

Tik Tak Admin — Kodun Tam İzahı (0-dan 100-ə)

Bu sənəd, layihədəki **hər bir faylı sətir-sətir** izah edir. Heç bir proqramlaşdırma təcrübəsi olmayan biri (kursu yeni başlayan səviyyəsi) belə, bu sənədi oxuyub layihənin necə işlədiyini başa düşə bilməlidir.

Necə oxumaq lazımdır: Əvvəlcə "Hissə 1" və "Hissə 2"-ni oxuyun (bunlar bütün kodda təkrar-təkrar rast gələcəyiniz JavaScript/React sintaksisini izah edir). Sonra istənilən sırayla, istədiyiniz faylın izahına keçə bilərsiniz — hər bölmə özbaşındır.

Mündəricat

- [Bu layihə nədir və nə üçün belə qurulub](#)
- [JavaScript/React sintaksis lüğəti — əvvəlcə bunu oxuyun](#)
- [Qovluq strukturu — hər qovluq nə üçündür](#)
- [Giriş nöqtəsi: `main.jsx` və `App.jsx`](#)
- [Marşrutlaşdırma \(Routing\): `AppRoutes.jsx`, `RequireAuth.jsx`, `RedirectIfAuth.jsx`](#)
- [Autentifikasiya \(Auth\): `session.js`, `useAuthStore.js`, `authService.js`](#)
- [API qatı: `axiosInstance.js` və servis faylları](#)
- [Adapterlər: API formatını UI formatına çevirmək](#)
- [Sabitlər \(Constants\): `productTypes.js`, `orderStatus.js`](#)
- [`formatDate.js`](#)
- [TanStack Query qatı: `queryClient.js`](#)
- [Ortaq \(shared\) komponentlər](#)
- [Öz hook-larımız \(custom hooks\)](#)
- [`Pagination.jsx`](#)
- [Layout: `AdminLayout.jsx`, `Sidebar.jsx`, `Header.jsx`](#)
- [Səhifələr: `Login`, `NotFound`, `Categories`, `Campaigns`, `Products`, `Orders`, `Users`](#)
- [CSS Modules necə işləyir](#)
- [Lüğət — tez-tez rast gələcəyiniz sözlər](#)

Hissə 1: Bu layihə nədir

Bu, **Tik Tak** adlı bir e-ticarət saytının **admin panelidir** — yəni mağazanın işçilərinin sifarişlərə, məhsullara, kateqoriyalara, kampaniyalara və istifadəçilərə baxıb idarə etdiyi daxili veb-tətbiqdır. Müştərilərin özlərinin gördüyü sayt deyil, "arxa ofis" hissəsidir.

Texnologiyalar:

- React** — istifadəçi interfeysini (UI) qurmaq üçün JavaScript kitabxanası. "Komponent" adlanan kiçik hissələrdən (düymə, cədvəl, forma) böyük səhifələr qururuq.
- Vite** — layihəni sürətlə işə salan və "build" (yəni brauzerin başa düşəcəyi fayllara çevirən) alət.
- react-router-dom** — brauzerdə ünvan çubuğundakı yol (`/sifarişlər` , `/kateqoriyalar` və s.) dəyişəndə hansı komponentin göstəriləcəyini idarə edir.
- axios** — backend (server) ilə HTTP sorğuları (GET, POST, PUT, DELETE) göndərmək üçün kitabxana.
- @tanstack/react-query** — serverdən gələn datanı (kateqoriyalar siyahısı, sifarişlər və s.) yükləmək, yadda saxlamaq (cache) və yeniləmək üçün kitabxana.
- zustand** — komponentlər arasında paylaşılan sadə "qlobal state" (bu layihədə yalnız login məlumatı üçün istifadə olunur).
- sonner** — ekranın küncündə çıxan bildiriş qutucuqları (toast) üçün kitabxana.
- lucide-react** — bütün ikonların (zəng, göz, qələm və s.) gəldiyi kitabxana.

- **CSS Modules** — hər komponentin öz CSS faylı olur və klaslar avtomatik unikal edilir ki, bir komponentin stili başqasına təsir etməsin.

Hissə 2: Sintaksis lüğəti

Kodun içində dəfələrlə görəəcəyiniz "qəliz görünən" simvolları burda sadələşdirib izah edirik. Bunları bir dəfə anlasanız, aşağıdakı bütün fayllar çox rahat oxunacaq.

import / export — fayllar arası əlaqə

```
import { useState } from 'react'      // "react" paketindən useState adlı şeyi gətir
import Button from '@shared/Button/Button' // Button faylından "default export"-u gətir
export default function App() { ... } // bu faylın ƏSAS (default) ixracı
export const foo = () => {}            // bu faylın ADI ilə ixrac olunan şeyi (bir fayldan
// bir neçə belə ola bilər)
```

- `export default` — hər faylda **yalnız bir dəfə** ola bilər, import edərkən istənilən adla çağırılabilir.
- `export const X` (adlı/named export) — bir fayldan bir neçə ola bilər, import edərkən **dəqiq həmin adla**, fərqli mötərizədə gətirilməlidir: `import { X } from '...'`.
- `@/` — bu layihədə "qısayoldur", `src/` qovluğuna işarə edir (`vite.config.js` -də təyin olunub). Yəni `@/shared/Button/Button` = `src/shared/Button/Button.jsx`.

Dəyişən elan etmək: `const` və `let`

```
const x = 5 // x-ə BİR DƏFƏ dəyər verilir, sonra dəyişdirilə bilməz
let y = 5   // y-ə sonra yenidən dəyər verilə bilər (y = 10)
```

Bu kodda demək olar həmişə `const` görəcəksiniz — funksiyalar da, obyektlər də adətən `const` -a yazılır.

Ox funksiyası (arrow function)

```
function topla(a, b) { return a + b } // klassik funksiya
const topla = (a, b) => a + b         // eyni şey, "ox funksiyası" forması
const salamlar = () => { console.log('salam') } // parametr yoxdursa boş mötərizə
const kvadrat = (x) => x * x           // { return ... } yazmasan, avtomatik "return" olur
```

(parametrlər) => nəticə — bu, "bu parametrləri al, bu nəticəni qaytar" deməkdir. Kodun demək olar hər yerində, xüsusən `.map()`, `.filter()`, `onClick={() => ...}` daxilində istifadə olunur.

Destructuring (obyektdən/massivdən "çıxarılmaq")

```
const user = { name: 'Ali', age: 20 }
const { name, age } = user // user.name-i `name`-ə, user.age-i `age`-ə çıxarır
const { name: adı } = user // "name" sahəsini fərqli adla (adı) çıxarmaq

const arr = [1, 2, 3]
const [birinci, ikinci] = arr // birinci=1, ikinci=2
```

```
function Button({ variant, children }) { ... } // funksiyanın PARAMETRİ birbaşa destructure olunur –  
  
// yəni <Button  
variant="solid">mətn</Button> çağırılanda  
  
// props obyektindən variant və children  
avtomatik çıxarılır
```

Spread (...) və Rest (...)

Eyni üç nöqtə, İKİ FƏRQLİ məna daşıyır, kontekstdən asılıdır:

```
// SPREAD – obyekt/massivi "açıb" başqasının içinə tökmək  
const a = { x: 1, y: 2 }  
const b = { ...a, z: 3 } // b = { x: 1, y: 2, z: 3 } (a-nın BÜTÜN sahələrini kopyalayır)  
const c = { ...a, x: 99 } // c = { x: 99, y: 2 } (sonra yazılan sahə əvvəlkini "üstələyir")  
  
const arr1 = [1, 2]  
const arr2 = [...arr1, 3] // arr2 = [1, 2, 3]  
  
// REST – "qalanları" bir yerə yığmaq (adətən funksiya parametrində)  
function Button({ variant, children, ...rest }) { ... }  
// variant və children ayrıca çıxarılır, QALAN bütün prop-lar (onClick, disabled, type və s.)  
// "rest" adlı BİR obyektə yığılır – sonra <button {...rest}> ilə hamısı birdən elementə verilir
```

Template literal (backtick ` ilə string)

```
const ad = 'Əli'  
const salam = `Salam, ${ad}!` // "Salam, Əli!" – ${...} içinə DƏYİŞƏN yazmaq olar  
const url = `/admin/categories/${id}` // dinamik URL qurmaq üçün çox işlədilir
```

Adi '...' və ya "..." daxilində \${} işləmir — yalnız backtick (`) daxilində işləyir.

Ternar operator (qısa if/else)

```
const mesaj = yaş >= 18 ? 'Böyüksən' : 'Kiçiksən'  
// oxunuşu: "əgər (yaş >= 18) DOĞRUDURSA → 'Böyüksən', YOXSA → 'Kiçiksən'  
  
{loading ? 'Göndərilir...' : 'Yarat'} // JSX daxilində şərtə görə mətn göstərmək
```

&& ilə şərti göstərmək (JSX-də çox işlədilir)

```
{loading && <Loading />}
```

Bu, "əgər `loading` doğrudursa, `<Loading/>` -u göstər, yoxsa HEÇ NƏ göstərmə" deməkdir. JavaScript-də `&&` soldan sağa yoxlayır — sol tərəf `false` olsa, sağ tərəfə belə baxmır (React da o zaman heç nə render etmir).

Optional chaining `?.` — "əgər varsa, daxil ol"

```
const ad = user?.profile?.name
// user undefined/null-dursa, XƏTA VERMƏDƏN sadəcə `undefined` qaytarır
// user varsa amma profile yoxdursa, yenə xəta vermir, undefined qaytarır
// YALNIZ user.profile.name-in HAMISI mövcud olanda əsl dəyəri qaytarır

item.category?.name ?? '' // aşağıda izah olunan ?? ilə birlikdə tez-tez görəcəksiniz
```

Bunsuz, `user.profile.name` yazsaydınız və `user` `null` / `undefined` olsaydı, proqram **xəta ilə dayanardı** ("Cannot read property 'profile' of undefined"). `?.` bunun qarşısını alır.

Nullish coalescing `??` — "yoxdursa, bunu istifadə et"

```
const say = itemCount ?? 0
// itemCount `null` və ya `undefined`-dursa → 0 istifadə olunur
// itemCount 0-dırsa (əsl rəqəm kimi) → 0 elə YENƏ 0 qalır (bu, || -dan FƏRQLİDİR!)
```

Diqqət: `??` yalnız `null` / `undefined` üçün işləyir. `||` isə `0`, `''`, `false` kimi bütün "falsy" dəyərləri də əvəz edir — bu kodda hər ikisi işlədilir, məqsədə görə seçilib (məs. `form.imageUrl || ''` — boş string istəyəndə `||` düzgündür).

Massiv metodları: `.map()`, `.filter()`, `.find()`, `.slice()`

```
const nömrələr = [1, 2, 3]

nömrələr.map(n => n * 2) // [2, 4, 6] – HƏR elementi çevirib YENİ massiv qaytarır
nömrələr.filter(n => n > 1) // [2, 3] – şərtə uyan elementləri SEÇİB yeni massiv qaytarır
nömrələr.find(n => n === 2) // 2 – şərtə uyan İLK elementi (təkcə birini) qaytarır, tapmasa `undefined`
nömrələr.slice(0, 2) // [1, 2] – indeks 0-dan 2-yə QƏDƏR (2 daxil deyil)
"kəsib" yeni massiv qaytarır
```

Bu layihədə `.map()` ən çox JSX daxilində siyahı göstərmək üçün işlədilir:

```
{categories.map((item) => <tr key={item.id}>{item.name}</tr>)}
```

Hər `item` üçün bir `<tr>` yaradır. `key={item.id}` React-a "bu sətir hansı data ilə bağlıdır" deyir ki, siyahı dəyişəndə düzgün yenilənsin.

`async / await` və `Promise` — "gözlə, nəticə gələne qədər"

Serverə sorğu göndərmək DƏRHAL bitmir (bir neçə millisaniyə çəkir). JavaScript bunu **Promise** (vəd) adlanan bir obyektlə idarə edir — "bu iş gələcəkdə bitəcək, bitəndə sənə xəbər verəcəm" deməkdir.

```
// .then() forması:
fetch('/api/data').then((cavab) => {
  console.log(cavab) // sorğu bitəndə bura işə düşür
})

// async/await forması (eyni məna, daha oxunaqlı):
async function məlumatAl() {
  const cavab = await fetch('/api/data') // bu sətirdə DAYANIR, sorğu bitənə qədər gözləyir
  console.log(cavab) // sonra buraya keçir
}
```

- **async** — funksiyanın daxilində **await** istifadə edəcəyini bildirir.
- **await** — "bu Promise bitənə qədər gözlə, sonra nəticəni bu dəyişənə yaz".
- **try { ... } catch (err) { ... }** — **await** -lə gözlədiyiniz iş XƏTA versə (məsələn, server 400/500 qaytarsa), program dayanmır, **catch** blokuna keçir, orada xətanı idarə edə bilərsiniz.

```
try {
  await createCategory(payload) // uğurlu olarsa davam edir
} catch (err) {
  toast.error(err.message) // xəta olarsa buraya düşür
}
```

React-in özü: komponent, props, JSX

```
function Salamlə({ ad }) { // bu bir KOMPONENTDİR – böyük hərflə başlayan funksiya
  return <h1>Salam, {ad}!</h1> // JSX – HTML-ə bənzəyən, əslində JavaScript olan
                                sintaksis
}

<Salamlə ad="Əli" /> // istifadəsi – "ad" adlı PROP verilir
```

- **Komponent** = UI-nin bir hissəsini qaytaran funksiya.
- **Props** = komponentə kənardan verilən "parametrlər" (`<Button variant="solid">` — **variant** bir prop-dur).
- **JSX** = `<div>...</div>` kimi görünən, amma əslində arxa planda `React.createElement(...)` çağırışlarına çevrilən sintaksis. JSX daxilində `{}` yazsanız, içində "təmiz JavaScript" işlədə bilərsiniz (`{ad}` , `{say + 1}` , `{items.map(...)}`).

useState — komponentin "yaddaşı"

```
const [say, setSay] = useState(0)
// say → CARİ dəyər (başlanğıcda 0)
// setSay → say-ı DƏYİŞMƏK üçün funksiya
// setSay(5) çağırırsanız, React komponenti YENİDƏN render edir, say artıq 5 olur

<button onClick={() => setSay(say + 1)}>Bas: {say}</button>
```

`useState` -siz, komponent daxilindəki adi dəyişənlər hər render-də "sıfırlanır" və ekranda dəyişiklik göstərmir.

`useState` React-a "bu dəyər render-lər arasında yadda saxla və dəyişəndə ekranı yenilə" deyir.

`useEffect` — "yan təsir" (side effect)

```
useEffect(() => {
  document.title = 'Yeni başlıq' // bu, komponent EKRANA ÇIXANDA (mount) işə düşür

  return () => {
    document.title = 'Köhnə başlıq' // bu işə komponent EKRANDAN GEDƏNDƏ (unmount) işə düşür
  }
}, []) // [] boş massiv = "yalnız BİR DƏFƏ, mount olanda işə sal"
```

`useEffect` -in ikinci parametri ([], [title], [open, onClose] və s.) "**dependency array**" adlanır — siyahıdakı dəyərlərdən HƏR HANSI biri dəyişəndə effekt YENİDƏN işə düşür. Boş [] = yalnız ilk dəfə. Heç yazmasanız = HƏR render-də (nadir hallarda istifadə olunur).

`useMemo` — hesablamayı "yaddan çıxarmamaq" (cache)

```
const filtered = useMemo(
  () => items.filter((i) => i.name.includes(search)), // bu hesablama
  [items, search], // yalnız bunlar dəyişəndə TƏKRAR işə düşür
)
```

`useMemo` -suz, komponent HƏR render-də (məsələn, başqa bir düymə klikləndə) bu filtri YENİDƏN hesablayardı, hətta `items` / `search` dəyişməsə belə. `useMemo` "əgər asılılıqlar eyni qalıbsa, köhnə nəticəni saxla, təzədən hesablama" deyir.

Custom hook — öz `useXxx()` funksiyanız

Adı `use` ilə başlayan İSTƏNİLƏN funksiya "hook" sayılır. Bunun daxilində `useState` / `useEffect` kimi hazır hook-ları işlədib, öz təkrarlanan məntiqinizi bir yerə yığa bilərsiniz (bu layihədə `usePagination`, `useCrudModal`, `useTitle`, `useDebounce` — hamısı buna misaldır, aşağıda ətraflı izah olunub).

CSS Modules idxalı

```
import styles from './Button.module.css'
// styles = { btn: 'Button-module__btn__aB3xZ', solid: 'Button-module__solid__k9F2p', ... }

<button className={styles.btn}>...</button>
```

`.module.css` şəklində bitən fayllar CSS Modules sayılır — Vite onları avtomatik tanıyır, hər klas adını UNİKAL edir (`.btn` → `Button-module__btn__aB3xZ` kimi bir şeyə çevrilir) ki, başqa faylda da `.btn` yazsanız TOQQUŞMA olmasın. `styles` obyektı CSS faylindəki klas adlarını JavaScript-dən əlçatan edir.

Hissə 3: Qovluq strukturu

src/	
├ app/	→ Tətbiqin "işə düşmə nöqtəsi" (main.jsx, App.jsx)
├ routes/	→ Marşrutlaşdırma (hansı URL-də hansı səhifə, giriş qoruması)
├ layouts/	→ Bütün admin səhifələrini əhatə edən "çərçivə" (sidebar + header)
├ components/	→ Yalnız layout-a aid komponentlər (Sidebar, Header)
├ pages/	
│ └ Login/	→ Giriş səhifəsi (qorunmayıb, hamı görə bilər)
│ └ NotFound/	→ 404 səhifəsi
│ └ protected/	→ Login tələb edən 5 səhifə (Orders, Campaigns, Categories, Products, Users)
├ shared/	→ Hər yerdə təkrar istifadə olunan UI hissələri (Button, Modal, Table, ...)
├ services/	→ Backend-ə HTTP sorğusu göndərən funksiyalar (axios ilə)
├ store/	→ Zustand ilə global state (yalnız auth üçün)
├ lib/	
│ └ adapters/	→ API formatını UI formatına (və əksinə) çevirən funksiyalar
│ └ auth/	→ localStorage-da token saxlamaq/oxumaq
│ └ constants/	→ Sabit dəyərlər (enum → Azərbaycan dilində etiket xəritələri)
│ └ queryClient.js	→ TanStack Query-nin konfigurasiyası
├ utils/	→ `Pagination` komponenti və `formatDate` funksiyası
├ assets/	→ Şəkillər (SVG-lər)
└ index.css	→ Bütün rənglər/ölçülər üçün CSS dəyişənləri (design token-lar)

Hissə 4: Giriş nöqtəsi

src/app/main.jsx

Bu, tətbiqin **HƏR ŞEYDƏN ƏVVƏL** işə düşən faylıdır — `index.html` faylı birbaşa bunu `<script>` ilə çağırır.

```
import { createRoot } from 'react-dom/client'
import '@index.css'
import ErrorBoundary from '@shared/ErrorBoundary/ErrorBoundary'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <ErrorBoundary>
    <App />
  </ErrorBoundary>,
)
```

Sətir-sətir:

- `createRoot` — React-in DOM-a (brauzerin göstərdiyi HTML ağacına) "bağlanmaq" üçün funksiyası.
- `import '@index.css'` — bütün global CSS-i (rəng dəyişənləri, font, reset qaydaları) bir dəfə yükləyir. `from` yoxdur çünki bu faylın heç nəyini "adla" gətirmirik, sadəcə CSS-i işə salırıq.
- `ErrorBoundary` — aşağıda ətraflı izah olunan xüsusi komponent, tətbiqin İSTƏNİLƏN yerində JavaScript xətası baş versə, bütün ekranı "ağappaq boş" etmək əvəzinə səliqəli bir xəta ekranı göstərir.
- `App` — `./App.jsx` -dən (eyni qovluqdan, ona görə `@/` yox, `./` işlədilib) gətirilir.
- `document.getElementById('root')` — `index.html` -dəki `<div id="root"></div>` elementini tapır — React BÜTÜN tətbiqi bunun İÇİNƏ "yerləşdirəcək".

6. `createRoot(...).render(...)` — tapılan `<div>` -in içinə `<ErrorBoundary><App/>`
`</ErrorBoundary>` -ni RENDER edir (ekrana çıxarır).

Niyə `<StrictMode>` yoxdur? React-in default şablonunda adətən `<StrictMode>` olur (development zamanı bəzi funksiyaları TƏSADÜFƏN İKİ DƏFƏ işə salıb səhvləri tez tapmağa kömək edir). Bu layihədə bilərəkdən çıxarılıb — sadəlik üçün.

Niyə `ErrorBoundary` `App` -in İÇİNDƏ deyil, ÇÖLÜNDƏ? Çünki `App.jsx` -in özündə də (məsələn provider-lərin qurulmasında) nəzəri cəhətdən xəta ola bilər — `ErrorBoundary` ən XARİCDƏ olsa, HƏR ŞEYİ (`App`-in özü daxil) əhatə edir.

`src/app/App.jsx`

```
import { BrowserRouter } from 'react-router-dom'
import { QueryClientProvider } from '@tanstack/react-query'
import { Toaster } from 'sonner'
import { queryClient } from '@lib/queryClient'
import AppRoutes from '@routes/AppRoutes'

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <AppRoutes />
        <Toaster position="top-right" richColors />
      </BrowserRouter>
    </QueryClientProvider>
  )
}
```

Sətir-sətir:

1. `BrowserRouter` — `react-router-dom`-un əsas "sarğı" komponentidir, brauzerin ünvan çubuğunu izləyib React-a bildirir.
2. `QueryClientProvider` — TanStack Query-ni bütün tətbiqə "tanıtmaq" üçün. Bunun `client` prop-una `queryClient` (aşağıda izah olunan konfigurasiya obyektini) verilir. Bu olmadan heç bir səhifədə `useQuery` / `useMutation` işləməz.
3. `Toaster` — `sonner` kitabxanasının bildiriş qutucuqlarını EKRANA ÇIXARAN komponent. `position="top-right"` — sağ yuxarı küncdə çıxsın. `richColors` — uğur/xəta üçün rəngli (yaşıl/qırmızı) fon versin.
4. **Diqqət:** `<AppRoutes/>` və `<Toaster/>` `<BrowserRouter>` -in İÇİNDƏDİR, amma bir-birinin QARDAŞıdır (biri digərinin içində deyil) — ikisi də eyni səviyyədə, yan-yanı render olunur.

App komponentinin `export default` olması — deməli, `main.jsx` -də `import App from './App.jsx'` yazanda, `App.jsx` -in İXRAC ETDİYİ İSTƏNİLƏN adı (`App`, `Foo`, nə olur olsun) bu adla gətirə bilərik, çünki `default` ixracın "adı" YOXDUR, gətirilən yerdə istənilən ad qoyula bilər.

Hissə 5: Routing

`src/routes/AppRoutes.jsx`


```

import { lazy, Suspense } from 'react'
import { Routes, Route, Navigate } from 'react-router-dom'
import RequireAuth from '@routes/RequireAuth'
import RedirectIfAuth from '@routes/RedirectIfAuth'
import AdminLayout from '@layouts/AdminLayout'
import Loading from '@shared/Loading/Loading'

const Login = lazy(() => import('@pages/Login/Login'))
const NotFound = lazy(() => import('@pages/NotFound/NotFound'))
const Orders = lazy(() => import('@pages/protected/Orders/Orders'))
const Campaigns = lazy(() => import('@pages/protected/Campaigns/Campaigns'))
const Categories = lazy(() => import('@pages/protected/Categories/Categories'))
const Products = lazy(() => import('@pages/protected/Products/Products'))
const Users = lazy(() => import('@pages/protected/Users/Users'))

export default function AppRoutes() {
  return (
    <Suspense fallback={<Loading fullScreen />}>
      <Routes>
        <Route
          path="/login"
          element={
            <RedirectIfAuth>
              <Login />
            </RedirectIfAuth>
          }
        />
        <Route element={<RequireAuth />}>
          <Route element={<AdminLayout />}>
            <Route path="/sifarisler" element={<Orders />} />
            <Route path="/kampaniyalar" element={<Campaigns />} />
            <Route path="/kateqoriyalar" element={<Categories />} />
            <Route path="/mehsullar" element={<Products />} />
            <Route path="/istifadeciler" element={<Users />} />
          </Route>
        </Route>
        <Route path="/" element={<Navigate to="/sifarisler" replace />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </Suspense>
  )
}

```

Nə üçün `lazy(() => import(...))` ? Normal `import Login from '...'` DƏRHAL, tətbiq açılan kimi o faylın kodunu yükləyir — hətta istifadəçi `/login` -ə heç getməsə belə. `lazy()` isə "bu komponentin kodunu YALNIZ LAZIM OLANDA (həmin route-a keçiləndə) yüklə" deyir. Nəticədə hər səhifə öz kiçik JS faylı (chunk) kimi ayrılır, ilk yükləmə daha sürətli olur.

Suspense nədir? `lazy()` ilə yüklənən komponentin kodu HƏLƏ GƏLMƏYİBSƏ (yüklənməsi bir neçə millisaniyə çəka bilər), React nə göstərəcəyini bilmir — buna görə `<Suspense fallback={...}>` "bu komponent hazır olana

qədər `fallback` -ı göstər" deyir. Burda `fallback={<Loading fullScreen/>}` — bütün ekranı ortalanmış spinner tutur.

Sətir-sətir marşrut ağacı:

- `<Route path="/login" element={<RedirectIfAuth><Login/></RedirectIfAuth>} />` — `/login` ünvanına gedəndə, əvvəlcə `RedirectIfAuth` yoxlayır (aşağıda izah), sonra `Login` -i göstərir.
- `<Route element={<RequireAuth/>}>` — bu, "layout route"-dur, öz `path` -ı yoxdur, sadəcə İÇİNDƏKİ bütün route-ları `RequireAuth` yoxlamasından keçirir (login olmayıbsa, heç birinə keçid vermir).
- Onun İÇİNDƏ daha bir `<Route element={<AdminLayout/>}>` var — bu da eyni məntiqə, İÇİNDƏKİ 5 səhifəni `AdminLayout` -un (sidebar+header) İÇİNƏ "yerləşdirir" (aşağıda `<Outlet/>` ilə izah olunur).
- `<Route path="/" element={<Navigate to="/sifarisler" replace/>} />` — kimsə sadəcə sayt adının kök ünvanına (/) girsə, avtomatik `/sifarisler` -ə YÖNLƏNDİRİLİR. `replace` — brauzerin "geri" düyməsi bu addımı ATLAYIR (tarixçəyə əlavə olunmur).
- `<Route path="*" element={<NotFound/>} />` — * "başqa HEÇ NƏYƏ uymayan İSTƏNİLƏN yol" deməkdir — yəni tanış olmayan bir URL yazılısa, 404 səhifəsi göstərilir.

src/routes/RequireAuth.jsx

```
import { Navigate, Outlet } from 'react-router-dom'
import { useAuthStore } from '@store/useAuthStore'

export default function RequireAuth() {
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated)
  return isAuthenticated ? <Outlet /> : <Navigate to="/login" replace />
}
```

Sətir-sətir:

1. `useAuthStore((s) => s.isAuthenticated)` — Zustand store-dan YALNIZ `isAuthenticated` sahəsini "seçib" alır (aşağıda Zustand-ı ətraflı izah edəcəyik). `(s) => s.isAuthenticated` bir "selector" funksiyasıdır — "store-un tam vəziyyətindən (s) mənə YALNIZ bu sahəni ver" deməkdir.
2. `isAuthenticated ? <Outlet/> : <Navigate to="/login" replace/>` — ternar operator ilə: login olubsa `<Outlet/>` göstər, olmayıbsa `/login` -ə göndər.
3. `<Outlet/>` **nədir?** React-router-dom-un xüsusi komponentidir — "burada, bu layout route-un İÇİNDƏKİ konkret route (uşaq route) render olunmalıdır" yer tutucusudur. `AppRoutes.jsx` -də `<Route element={<RequireAuth/>}>` -nin İÇİNDƏKİ hər route (`/sifarisler` , `/kampaniyalar` və s.) məhz bu `<Outlet/>` -in yerinə "yerləşdirilir".

src/routes/RedirectIfAuth.jsx

```
import { Navigate } from 'react-router-dom'
import { useAuthStore } from '@store/useAuthStore'

export default function RedirectIfAuth({ children }) {
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated)
  return isAuthenticated ? <Navigate to="/sifarisler" replace /> : children
}
```

`RequireAuth` -in TƏRSİDİR: artıq login OLMUŞ bir istifadəçi `/login` -ə girməyə cəhd etsə, ona login formasını göstərmək əvəzinə birbaşa `/sifarisler` -ə yönləndirir. `children` prop-u — `<RedirectIfAuth><Login/>`

`</RedirectIfAuth>` yazanda, `<Login/>` elementi `children` kimi ötürülür, biz onu login olmayanda sadəcə geri qaytarırıq (: children).

Hissə 6: Auth

`src/lib/auth/session.js`

Bu fayl, giriş məlumatlarını (token-lər, profil) brauzerin `localStorage` -ında saxlayır. `localStorage` — brauzerin diskində saxlanan, səhifə bağlanıb-açılsa belə İTMƏYƏN açar-dəyər (key-value) yaddaşdır.

```
const ACCESS_KEY = 'tiktak_admin_access_token'
const REFRESH_KEY = 'tiktak_admin_refresh_token'
const PROFILE_KEY = 'tiktak_admin_profile'

export function getAccessToken() {
  return localStorage.getItem(ACCESS_KEY)
}

export function getRefreshToken() {
  return localStorage.getItem(REFRESH_KEY)
}

export function getStoredProfile() {
  const raw = localStorage.getItem(PROFILE_KEY)
  return raw ? JSON.parse(raw) : null
}

export function saveSession({ tokens, profile }) {
  localStorage.setItem(ACCESS_KEY, tokens.access_token)
  localStorage.setItem(REFRESH_KEY, tokens.refresh_token)
  localStorage.setItem(PROFILE_KEY, JSON.stringify(profile))
}

export function saveTokens(tokens) {
  localStorage.setItem(ACCESS_KEY, tokens.access_token)
  localStorage.setItem(REFRESH_KEY, tokens.refresh_token)
}

export function clearSession() {
  localStorage.removeItem(ACCESS_KEY)
  localStorage.removeItem(REFRESH_KEY)
  localStorage.removeItem(PROFILE_KEY)
}
```

Sətir-sətir: 1-3. Üç sabit — `localStorage` -da istifadə ediləcək "açar adları". Bunları sabit dəyişənə çıxarmağın səbəbi: eyni sətir (`'tiktak_admin_access_token'`) 6 yerdə yazıb, bir yazı səhvi edəndə hər yerin sınmasının qarşısını almaqdır. 5-7. `getAccessToken()` — `localStorage.getItem(ACCESS_KEY)` sadəcə saxlanmış access token-i (string, ya da tapılmasa `null`) qaytarır. 9-11. `getRefreshToken()` — eynilə, refresh token üçün. 13-16. `getStoredProfile()` — `localStorage` HƏMİŞƏ **string** saxlayır, obyekt yox. Ona görə profil `JSON.stringify()` ilə YAZILIB, oxuyanda `JSON.parse()` ilə GERİ obyektə çevrilir. `raw ? JSON.parse(raw) :`

`null` — əgər `raw` (string) varsa parse et, yoxdursa (`null` -dursa) sadəcə `null` qaytar (parse etməyə çalışsa xəta verərdi). 18-22. `saveSession({ tokens, profile })` — login uğurlu olduqda ÇAĞIRILIR. Parametr birbaşa `destructure` olunub (`{ tokens, profile }`) — çağıran `saveSession({ tokens: {...}, profile: {...} })` formasında verir. Hər üç dəyəri `localStorage` -a yazır. 24-27. `saveTokens(tokens)` — YALNIZ token-ləri yeniləmək üçün (profil dəyişmir) — token "refresh" olunanda istifadə olunur (aşağıda `axiosInstance.js` -də). 29-33. `clearSession()` — logout-da bütün üç açarı `localStorage` -dan SİLİR.

src/store/useAuthStore.js

Bu, **zustand** ilə qurulmuş "qlobal state"-dir. Zustand-ı belə düşünün: `useState` yalnız BİR komponentin daxilində yaşayır, komponent yox olduqda dəyər də itir. Zustand store isə tətbiqin İSTƏNİLƏN yerindən əlçatandır və HƏMİŞƏ yaddadır (səhifə naviqasiyası zamanı sıfırlanmır).

```
import { create } from 'zustand'
import { loginAdmin } from '@services/authService'
import { getAccessToken, getStoredProfile, saveSession, clearSession } from
'@/lib/auth/session'

export const useAuthStore = create((set) => ({
  profile: getStoredProfile(),
  isAuthenticated: !!getAccessToken(),

  login: async (phone, password) => {
    const data = await loginAdmin({ phone, password })
    saveSession(data)
    set({ profile: data.profile, isAuthenticated: true })
  },

  logout: () => {
    clearSession()
    set({ profile: null, isAuthenticated: false })
  },
}))

window.addEventListener('storage', () => {
  useAuthStore.setState({
    isAuthenticated: !!getAccessToken(),
    profile: getStoredProfile(),
  })
})
```

Sətir-sətir:

1. `create` — `zustand`-ın store yaradan funksiyası.
2. `create((set) => ({ ... })))` — `create` -ə bir funksiya verilir, bu funksiya `set` adlı bir "dəyişdirici" alır və store-un İLKİN vəziyyətini + funksiyalarını qaytarır.
3. `profile: getStoredProfile()` — store YARADILAN ANDA (səhifə ilk açıldıqda) `localStorage` -dan profili oxuyub başlanğıc dəyər kimi qoyur. Beləliklə, səhifəni yeniləsəniz (F5) belə, login "yadda qalır".
4. `isAuthenticated: !!getAccessToken()` — `!!` iki dəfə "yox" (NOT) işarəsidir, İSTƏNİLƏN dəyəri `true` / `false` -a çevirmək üçün trik: `getAccessToken()` bir string ("...") ya da `null` qaytarır; `!null` → `true`, `!"..."` → `false`, sonra bir daha `!` vuraraq: `!!null` → `false`, `!!!"..."` → `true`. Yəni

"token varsa true , yoxdursa false ". 9-13. login — async funksiyadır (içində await var). loginAdmin (servis funksiyası, aşağıda) çağırılır, cavab gözlənilir, saveSession ilə localStorage -a yazılır, sonra set({...}) ilə store-un CARİ vəziyyəti YENİLƏNİR — bu, bütün useAuthStore -a abunə olmuş komponentləri (RequireAuth , Sidebar və s.) AVTOMATİK yenidən render etdirir. 15-18. logout — clearSession() ilə localStorage təmizlənir, set({...}) ilə store-da profile: null, isAuthenticated: false qoyulur. 21-27. **Tab-lar arası sinxronizasiya** — window.addEventListener('storage', ...) brauzerin xüsusi bir hadisəsinə (event) qulaq asır. storage event-i YALNIZ o zaman atılır ki, localStorage BAŞQA BİR TAB-DA dəyişsin (öz tab-ınızda dəyişəndə SİZDƏ atılmır, digər açıq tab-larda atılır). Yəni: bir tab-da "Çıxış" etsəniz, digər açıq tab bunu bu listener vasitəsilə eşidir və öz isAuthenticated -ini də false -a çevirir → o tab-dakı RequireAuth da dərhal /login -ə yönləndirir. 23-26. useAuthStore.setState({...}) — komponentin İÇİNDƏN çağırılan set funksiyasından FƏRQLİ olaraq, bu, store-u KOMPONENT OLMADAN, birbaşa store obyektinin özündən yeniləməkdir (bu sətir bir React komponenti daxilində deyil, faylın "kök" səviyyəsində yazılıb).

src/services/authService.js

```
import api from './axiosInstance'

export const loginAdmin = (payload) => api.post('/auth/admin/login', payload, {
  skipAuthRetry: true })
export const getProfile = () => api.get('/admin/profile')
```

Sətir-sətir:

1. api — axiosInstance.js -dən gətirilən, artıq konfigurasiya edilmiş axios "instansı" (obyekti). Adı axios yox, MƏHZ BU layihənin özəl ayarları (base URL, header-lər, xəta idarəsi) ilə olanı.
2. loginAdmin — api.post(yol, data, əlavə_ayarlar) . /auth/admin/login -ə payload ({phone, password}) POST edir. { skipAuthRetry: true } — bu, axios-un ÖZÜNÜN tanıyacağı bir seçim deyil, BİZİM ÖZ konfigurasiyamızda (axiosInstance.js -də) yoxlanan xüsusi bir bayraqdır: "bu sorğu 401 alsa, token-i yeniləməyə CƏHD ETMƏ" (çünki bu, MƏHZ LOGIN sorğusudur — səhv parol daxil edəndə 401 gəlir, bu, "sessiya bitib" demək deyil).
3. getProfile — sadə GET sorğusu, hazırda heç bir səhifədə İSTİFADƏ OLUNMUR, gələcək üçün hazırdır.

Hissə 7: API qatı

src/services/axiosInstance.js — ən mürəkkəb fayl, diqqətlə oxuyun

Bu fayl, layihədəki BÜTÜN backend sorğularının keçdiyi "mərkəzi məntəqədir". Hər api.get(...) / api.post(...) çağırışı, əslində bu fayldakı qaydalardan keçir.

```
import axios from 'axios'
import { getAccessToken, getRefreshToken, saveTokens, clearSession } from
'@/lib/auth/session'
import { useAuthStore } from '@/store/useAuthStore'

const BASE_URL = `${import.meta.env.VITE_API_BASE_URL}/api/tiktak`

const api = axios.create({
  baseURL: BASE_URL,
```

```
headers: { 'Accept-Language': 'az' },
})
```

Sətir-sətir: 5. `import.meta.env.VITE_API_BASE_URL` — Vite-in xüsusi sintaksisidir, `.env` faylındakı `VITE_API_BASE_URL=https://...` dəyərini oxuyur. `${...}` template literal-dır (yuxarıda izah olundu) — iki string-i BİRLƏŞDİRİR: `.env` -dəki ünvan + sabit `/api/tiktak` sonluğu. 7-10. `axios.create({...})` — YENİ, ÖZƏLLƏŞDİRİLMİŞ bir axios "instansı" yaradır (adi `axios` -dan fərqli olaraq). `baseUrl` — bundan sonra `api.get('/admin/categories')` yazsanız, əslində TAM ünvana (`BASE_URL` + `'/admin/categories'`) sorğu gedir. `headers: { 'Accept-Language': 'az' }` — HƏR sorğuya avtomatik bu header əlavə olunur (backend-ə "cavabı Azərbaycan dilində ver" demək üçün).

```
api.interceptors.request.use((config) => {
  const token = getAccessToken()
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

"Interceptor" nədir? — Hərfi tərcüməsi "ələ keçirən"dir. Axios-a "HƏR sorğu göndərilməzdən ƏVVƏL (request interceptor) BU funksiyani işə sal" deyir. Burada: hər sorğudan əvvəl `localStorage` -dan token oxunur, VARSA, sorğunun header-lərinə `Authorization: Bearer <token>` əlavə olunur. `config` — sorğunun bütün ayarlarını (url, method, header-lər) daşıyan obyektdir, funksiyanın SONUNDA MÜTLƏQ geri qaytarılmalıdır (`return config`), yoxsa sorğu GETMƏZ.

```
const handleSuccess = (response) => response.data.data ?? response.data
```

Backend HƏR cavabı `{ message: "Ok", data: {...}, result: true }` formasında qaytarır (bu, layihənin backend-inin öz qaydasıdır — `docs/API.md` -də sənədləşdirilib). Bizə isə YALNIZ `data` hissəsi lazımdır, `message` / `result` yox. `response.data.data` — bu, "cavabın body-sinin İÇİNDƏKİ `data` sahəsi" deməkdir (`response.data` = bütün body, `response.data.data` = onun `data` sahəsi). `?? response.data` — əgər `data` sahəsi YOXDURSA (bəzi endpoint-lər — məs. statistika — bu qaydaya uymur, birbaşa xam obyekt qaytarır), onda BÜTÜN body-ni qaytar.

```
const STATUS_MESSAGES = {
  400: 'Məlumatlar düzgün deyil',
  403: 'Bu əməliyyat üçün icazəniz yoxdur',
  404: 'Tapılmadı',
  409: 'Bu məlumat artıq mövcuddur',
  422: 'Məlumatlar düzgün deyil',
  500: 'Server xətası baş verdi',
}

function getErrorMessage(error, isLogin) {
  if (!error.response) return 'Serverə qoşulmaq mümkün olmadı'
  if (error.response.status === 401) {
    return isLogin ? 'Telefon və ya parol yanlışdır' : 'Sessiya bitib, yenidən daxil olun'
  }
  return STATUS_MESSAGES[error.response.status] || 'Xəta baş verdi'
}
```

`STATUS_MESSAGES` — bir OBYEKTDİR, "açarları" HTTP status kodlarıdır (400, 403 və s. — server sorğuya CAVAB olaraq qaytardığı rəqəm, "nə baş verdiyini" bildirir), dəyərləri isə Azərbaycan dilində mesajlardır. Backend-in ÖZÜ ingiliscə mesaj qaytarır ("Password is wrong!" kimi) — biz bunu İSTİFADƏÇİYƏ GÖSTƏRMİRİK, ƏVƏZİNƏ status koduna görə ÖZ Azərbaycanca mesajımızı seçirik ki, bütün bildirişlər eyni dildə olsun.

`getErrorMessage(error, isLogin) :`

- `if (!error.response) return '...'` — `error.response` YOXDURSA (server heç cavab verməyib — internet kəsilib, server düşüb), "serverə qoşulmaq mümkün olmadı" qaytarır.
- `if (error.response.status === 401)` — 401 = "icazən yoxdur" (token səhvdir/vaxtı keçib, ya da login-də parol səhvdir). `isLogin ? '...' : '...'` — ƏGƏR bu, login sorğusudursa (parametr olaraq ötürülür) "parol yanlışdır" göstərir, YOX ƏGƏR başqa bir sorğudursa "sessiya bitib" göstərir — çünki İKİ FƏRQLİ hadisədir.
- Son sətir: `STATUS_MESSAGES[error.response.status]` — obyektədən status koduna GÖRƏ mesajı ÇIXARIR (`[...]` ilə DİNAMİK açar oxumaq — `obj.key` ilə eyni məna, sadəcə açar bir DƏYİŞƏNDƏ olanda `[]` işlədilir). `|| 'Xəta baş verdi'` — obyektə HƏMİN kod ÜÇÜN mesaj yoxdursa (məs. 502), ÜMUMİ bir mesaj qaytarır.

```
let refreshPromise = null

function refreshAccessToken() {
  if (!refreshPromise) {
    refreshPromise = axios
      .post(`${BASE_URL}/auth/refresh`, { refresh_token: getRefreshToken() }, { headers: {
'Accept-Language': 'az' } })
      .then((res) => {
        saveTokens(res.data.data)
        return res.data.data
      })
      .finally(() => {
        refreshPromise = null
      })
  }
  return refreshPromise
}
```

Bu funksiya, access token-in VAXTI keçəndə YENİSİNİ almaq üçündür (backend-in `/auth/refresh` endpoint-i vasitəsilə).

- `let refreshPromise = null` — `const` YOX, `let` (çünki bu dəyər DƏYİŞƏCƏK). Başlanğıcda heç bir "davam edən refresh sorğusu" yoxdur.
- `if (!refreshPromise)` — ƏGƏR HAZIRDA davam edən bir refresh sorğusu YOXDURSA, YENİ bir sorğu BAŞLAT. **Niyə bu yoxlama var?** Təsəvvür edin ki, EYNİ ANDA 3 fərqli sorğu 401 alıb — hər üçü token-i yeniləməyə çalışsa, backend-ə 3 DƏFƏ refresh sorğusu gedərdi (lazımsız). Bu yoxlama ilə: birinci sorğu refresh-i BAŞLADIR VƏ `refreshPromise` -a YAZIR, digər ikisi "artıq davam edir" görüb, EYNİ `refreshPromise` -ı GÖZLƏYİR (aşağıda `return refreshPromise` buna görədir).
- `axios.post(...)` — DİQQƏT: `api.post` YOX, sadə `axios.post` ! Çünki `api` instansının ÖZÜNDƏ bizim interceptor-larımız var — əgər `api.post` işlətsəydik və bu sorğu DA 401 alsaydı, YENİDƏN `handleError` -a düşüb, YENİDƏN refresh cəhd edərdi — SONSUZ DÖVRƏ yaranardı. Sadə `axios.post` bu interceptor-ları BÜTÜNLƏYKƏN keçir.

- `.then((res) => { saveTokens(res.data.data); return res.data.data })` — sorğu uğurlu olanda, yeni token-ləri `localStorage` -a yazır VƏ onları qaytarır.
- `.finally(() => { refreshPromise = null })` — sorğu istər uğurlu, istər uğursuz bitsin, `finally` HƏR HALDA işə düşür — `refreshPromise` -ı yenidən `null` -a QAYTARIR ki, NÖVBƏTİ dəfə YENİ bir `refresh` cəhdi edilə bilsin.
- `return refreshPromise` — bu, EGƏR yeni sorğu başladılıbsa, ONU; YOX, ARTIQQ davam edən vardisa, O DAVAM edəni qaytarır.

```
const handleError = async (error) => {
  const original = error.config
  const isUnauthorized = error.response?.status === 401
  const canRetry = isUnauthorized && !original.skipAuthRetry && !original._retry &&
  getRefreshToken()

  if (canRetry) {
    original._retry = true
    try {
      await refreshAccessToken()
      return api(original)
    } catch {
      clearSession()
      useAuthStore.getState().logout()
    }
  } else if (isUnauthorized && !original.skipAuthRetry) {
    clearSession()
    useAuthStore.getState().logout()
  }

  return Promise.reject(new Error(getErrorMessage(error, original.skipAuthRetry)))
}

api.interceptors.response.use(handleSuccess, handleError)

export default api
```

Bu, "response interceptor"-un XƏTA hissəsidir — HƏR sorğu XƏTA (400, 401, 404, 500 və s.) ilə qayıdanda BU funksiya işə düşür.

- `const original = error.config` — XƏTA VERƏN ORİJİNAL sorğunun bütün ayarlarını (url, method, data) saxlayır — lazım olsa, EYNİ sorğunu TƏKRAR göndərmək üçün.
- `const isUnauthorized = error.response?.status === 401` — status kodu DƏQİQ 401-dirsə `true` .
?. — `error.response` YOXDURSA (şəbəkə xətasıdırsa) xəta VERMƏDƏN `undefined` qaytarır, `undefined === 401` isə `false` olur — düzgün işləyir.
- `const canRetry = isUnauthorized && !original.skipAuthRetry && !original._retry && getRefreshToken()` — DÖRD şərtin HAMISI doğru olmalıdır ki, `canRetry` `true` olsun:
 1. `isUnauthorized` — 401-dir,
 2. `!original.skipAuthRetry` — bu sorğu "retry etmə" işarəli DEYİL (login sorğusu belə işarələnib, yuxarıda görmüşdük),
 3. `!original._retry` — bu sorğu ARTIQQ BİR DƏFƏ retry EDİLMƏYİB (aşağıda `original._retry = true` yazılır ki, İKİNCİ DƏFƏ eyni sorğu YENƏ 401 versə, SONSUZ dövrəyə düşməsin),

4. `getRefreshToken()` — `localStorage` -da bir refresh token VAR (yoxdursa, refresh cəhd etməyin mənası yoxdur).
- `if (canRetry) { ... }` — bütün şərtlər ödənilibə:
 - `original._retry = true` — bu sorğunu "artıq cəhd edilib" kimi İŞARƏLƏYİR.
 - `await refreshAccessToken()` — YUXARIDA izah olunan funksiyanı çağırır, YENİ token gələnə qədər GÖZLƏYİR.
 - `return api(original)` — token yeniləndikdən SONRA, ORİJİNAL sorğunu (indi YENİ token ilə, çünki request interceptor YENİDƏN işə düşəcək) TƏKRAR göndərir.
 - `catch { ... }` — `refreshAccessToken()` DƏ uğursuz olsa (refresh token da etibarsızdırsa), `clearSession()` + `logout()` çağırılır — istifadəçi TAM çıxış edir.
 - `else if (isUnauthorized && !original.skipAuthRetry)` — ƏGƏR 401-dir AMMA retry ŞƏRTLƏRİ ödənmirsə (məs. refresh token YOXDUR) — birbaşa `logout`.
 - **Diqqət:** `original.skipAuthRetry` OLAN sorğular (login) BU BLOKLARIN HEÇ BİRİNƏ DÜŞMÜR — sadəcə aşağıya, son sətərə keçir.
 - `return Promise.reject(new Error(getErrorMessage(error, original.skipAuthRetry)))` — FUNKSİYANIN SONU: XƏTANİ Azərbaycanca mesajla YENİDƏN "rədd edir" (reject) — bu, çağırılan koda (`catch (err) { toast.error(err.message) }`) gedib çatır.
 - `api.interceptors.response.use(handleSuccess, handleError)` — BU SƏTİR, YUXARIDA yazılan İKİ funksiyanı FAKTİKİ OLARAQ AXIOS-A "TANIDIR": BİRİNCİ argument (`handleSuccess`) uğurlu cavablar üçün, İKİNCİ (`handleError`) xətlər üçün işə düşür.
 - `export default api` — KONFİQURASIYA OLUNMUŞ axios instansını İXRAC edir, BÜTÜN SERVİS fayllarında (`categoryService.js` və s.) BUNU istifadə edirik.

Servis faylları (`src/services/*.js`)

Bu fayllar ÇOX SADƏDİR — hər biri BİR resurs (kateqoriyalar, məhsullar və s.) üçün HTTP sorğusu FUNKSİYALARINI ixrac edir. Məsələn:

```
// categoryService.js
import api from './axiosInstance'

export const listCategories = () => api.get('/admin/categories')
export const createCategory = (payload) => api.post('/admin/category', payload)
export const updateCategory = (id, payload) => api.put(`/admin/categories/${id}`, payload)
export const deleteCategory = (id) => api.delete(`/admin/categories/${id}`)
```

- `api.get('/admin/categories')` — GET sorğusu, siyahını GƏTİRİR.
- `api.post('/admin/category', payload)` — POST, YENİ kateqoriya YARADIR (`payload` — göndəriləcək data).
- `api.put(`/admin/categories/${id}`, payload)` — PUT, MÖVCUD kateqoriyanı YENİLƏYİR (`id` şablon literalla URL-in İÇİNƏ yerləşdirilir).
- `api.delete(`/admin/categories/${id}`)` — DELETE, KATEQORIYANI SİLİR.

Diqqət: `createCategory` TƏK sayılı `/admin/category`, digər 3-ü isə CƏM sayılı `/admin/categories` yolundan istifadə edir — bu, BİZİM SƏHVİMİZ DEYİL, BACKEND-in ÖZ QAYDASIDIR (sənədləşdirilib, `docs/API.md` -yə baxın).

`orderService.js` bir az FƏRQLİDİR:

```
export const listOrders = () => api.get('/orders/admin')
export const getOrderStats = () => api.get('/orders/admin/stats')
```

```
export const updateOrderStatus = (id, status) => api.put(`/orders/admin/${id}/status`, {
  status })
```

`updateOrderStatus(id, status)` — İKİNCİ argument `payload` obyektı DEYİL, sadə bir `status` string-idir, AMMA PUT sorğusunun body-si HƏMİŞƏ OBYEKT olmalıdır, ona görə `{ status }` yazılıb — bu, `{ status: status }` -in QISA FORMASIDIR (JavaScript-də, obyekt daxilində açar VƏ dəyər eyni ADDIRSA, TƏKRAR yazmağa EHTİYAC yoxdur).

`uploadService.js` isə FƏRQLİ bir NÖV data göndərir:

```
export const uploadImage = (file) => {
  const formData = new FormData()
  formData.append('file', file)
  return api.post('/upload', formData, { headers: { 'Content-Type': 'multipart/form-data' } })
}
```

`FormData` — brauzerin daxili bir obyektidir, FAYL göndərmək üçün istifadə olunur (adi JSON YOX).

`.append('file', file)` ilə faylı bu obyektə ƏLAVƏ edir, sonra `Content-Type: multipart/form-data` header-i ilə göndərir (server FAYL sorğularını BELƏ gözləyir).

Hissə 8: Adapterlər

Niyə adapter lazımdır? Backend datanı BİR FORMATDA göndərir (`img_url` , `created_at` , `snake_case` sahə adları), amma UI-mizin (forma, cədvəl) İSTİFADƏ ETDİYİ sahə adları FƏRQLİDİR (`imageUrl` , `date`). Adapter faylları BU İKİ FORMAT arasında "tərcüməçi" rolunu OYNAYIR.

`src/lib/adapters/category.js`

```
import { formatDate } from '@/utils/formatDate'

const FALLBACK = { image: '👉', color: '#f3f4f6' }

export const mapCategoryFromApi = (c) => ({
  id: c.id,
  ...FALLBACK,
  imageUrl: c.img_url || '',
  name: c.name,
  description: c.description,
  date: formatDate(c.created_at),
})

export const mapCategoryToApi = (form) => ({
  name: form.name,
  description: form.description,
  img_url: form.imageUrl || '',
})
```

Sətir-sətir: 3. `FALLBACK` — API-dən HEÇ VAXT gəlməyən, amma UI-nin gözlədiyi "dekorativ" sahələr (emoji + fon rəngi) — API-də ŞƏKİL yoxdursa, BU EMOJİ göstərilir. 5-12. `mapCategoryFromApi(c)` — API-DƏN GƏLƏN xam obyekti (`c`) UI formatına ÇEVİRİR:

- `id: c.id` — dəyişmir, birbaşa köçürülür.
- `...FALLBACK` — `SPREAD` ilə `image / color` sahələrini BURAYA "tökür" (yuxarıda izah olundu).
- `imageUrl: c.img_url || ''` — API-nin `img_url` -unu UI-nin `imageUrl` -una köçürür (AD DƏYİŞİR!).
`|| ''` — `img_url` `null / undefined` /boş `STRİNGDİRSƏ`, boş `STRİNG` istifadə olunur (`undefined` YOX).
- `date: formatDate(c.created_at)` — API-nin ISO tarixini (`"2025-06-12T05:37:56.753Z"` kimi) `OXUNAQLI` formata (`"12.06.2025"`) ÇEVİRİR (aşağıda `formatDate` izah olunur). 14-18.
`mapCategoryToApi(form)` — TƏRS İSTİQAMƏT: `FORMA` datasını (UI formatı) API-nin GÖZLƏDİYİ formata ÇEVİRİR — YARADILAN/YENİLƏNƏN kateqoriya BUNUNLA serverə GÖNDƏRİLİR. `image / color / date` GÖNDƏRİLMİR (API bunları QƏBUL ETMİR), YALNIZ `name / description / img_url` .

`src/lib/adapters/order.js` (bir az daha MÜRƏKKƏBDİR)

```
import { formatDate } from '@/utils/formatDate'
import { PRODUCT_TYPE_LABELS } from '@/lib/constants/productTypes'

const FALLBACK = { image: '📦', color: '#f3f4f6' }

export const mapOrderFromApi = (o) => ({
  id: o.id,
  orderNumber: o.orderNumber,
  date: formatDate(o.createdAt),
  address: o.address,
  phone: o.phone,
  paymentMethod: o.paymentMethod === 'CARD' ? 'Kart' : 'Nağd',
  status: o.status,
  subtotal: o.total,
  freeShipping: Number(o.deliveryFee) === 0,
  itemCount: o.items?.length ?? 0,
  user: o.user ?? null,
  items: (o.items ?? []).map((it) => ({
    name: it.product?.title ?? '',
    category: it.product?.category?.name ?? '',
    weight: `${it.quantity} ${PRODUCT_TYPE_LABELS[it.product?.type] ?? ''}.trim(),
    price: it.product?.price ?? it.total_price,
    unit: PRODUCT_TYPE_LABELS[it.product?.type] ?? '',
    ...FALLBACK,
  })),
}))
```

Fərqli/mürəkkəb sətirlər:

- `paymentMethod: o.paymentMethod === 'CARD' ? 'Kart' : 'Nağd'` — API-nin ingiliscə enum-unu (`"CARD"` / `"CASH"`) Azərbaycanca sözə ÇEVİRİR.
- `freeShipping: Number(o.deliveryFee) === 0` — `deliveryFee` (çatdırılma haqqı) BİR `STRİNGDİRSƏ` (`"0.00"`), `Number(...)` onu ƏSL RƏQƏMƏ çevirir, sonra `=== 0` yoxlayır — "haqq sıfırıdırsa, pulsuz çatdırılmadır" (`true / false`).

- `itemCount: o.items?.length ?? 0` — `o.items` bir MASSİVDİRSƏ, `.length` -i (say) götürür; `o.items` `null` / `undefined` -dirsə `?.` sayəsində XƏTA VERMİR, sonra `?? 0` ilə "0" DEFAULT DƏYƏRİ QOYULUR.
- `items: (o.items ?? []).map((it) => ({ ... }))` — `o.items` YOXDURSA, BOŞ MASSİV (`[]`) istifadə olunur (ki, `.map()` XƏTA VERMƏSİN), sonra HƏR bir sifariş ELEMENTİNİ (`it`) UI FORMATINA çevirir:
 - `name: it.product?.title ?? ''` — MƏHSULUN adı (backend-də `title` adlanır) — `?.` ilə `it.product` YOXDURSA XƏTA VERMİR.
 - `weight: \ ${it.quantity} ${PRODUCT_TYPE_LABELS[it.product?.type] ?? ''}.trim()` — MİQDAR + ölçü VAHİDİNİ ("2 ədəd" kimi) BİRLƏŞDİRİR. `PRODUCT_TYPE_LABELS[...]` — sabitlər faylından (aşağıda) "kg" → "Kiloqram" kimi ETİKET oxuyur. `.trim()` — nəticənin ƏVVƏLİNDƏ/SONUNDA yaranan boşluqları TƏMİZLƏYİR (məs. ölçü tapılmasa boş qalar, ətrafda boşluq qalmasın deyə).
 - `...FALLBACK` — HƏR ELEMENTƏ dekorativ emoji+rəng ƏLAVƏ EDİR (sifariş elementlərinin ÖZ şəkli YOXDUR).

src/lib/adapters/product.js

```
export const mapProductFromApi = (p) => ({
  id: p.id,
  ...FALLBACK,
  imageUrl: p.img_url || '',
  name: p.title, // DİQQƏT: API "title" adlanır, UI "name" işlədir!
  description: p.description,
  price: p.price,
  type: p.type,
  category: p.category ?? null, // NESTED (iç-içə) obyekt — {id, name, ...}
  category_id: p.category?.id ?? '', // dropdown üçün, TƏK ID lazımdır
  date: formatDate(p.created_at),
})

export const mapProductToApi = (form) => ({
  title: form.name, // GERİYƏ "title"-a çevrilir
  description: form.description,
  price: String(form.price), // `String(...)` rəqəmi/nə olur-olsun STRİNGƏ ÇEVİRİR
  type: form.type,
  img_url: form.imageUrl || '',
  category_id: Number(form.category_id), // `Number(...)` STRİNGİ RƏQƏMƏ çevirir (HTML
  // select həmişə STRİNG qaytarır)
})
```

`String(form.price)` və `Number(form.category_id)` — HTML formalarındakı DƏYƏRLƏR həmişə STRİNGDİR (hətta `<input type="number">` olsa belə), amma API MÜƏYYƏN sahələrdə DƏQİQ TİP (rəqəm) GÖZLƏYİR — buna görə GÖNDƏRMƏZDƏN ƏVVƏL AÇIQ ÇEVİRİRİK.

src/lib/adapters/user.js (ən SADƏSİ)

```
export const mapUserFromApi = (u) => ({
  id: u.id,
```

```

initial: (u.full_name || '?').charAt(0).toUpperCase(),
color: '#22c55e',
name: u.full_name,
phone: u.phone,
address: u.address || 'Qeyd olunmayıb',
role: u.role,
})

```

`initial: (u.full_name || '?').charAt(0).toUpperCase()` — İSTİFADƏÇİNİN adının BİRİNCİ HƏRFİNİ (avatar üçün) ÇIXARIR: `u.full_name || '?'` (ad YOXDURSA "?" işarəsi), `.charAt(0)` (BİRİNCİ SİMVOLU götürür), `.toUpperCase()` (BÖYÜK hərfə çevirir). Bu faylda `mapUserToApi` YOXDUR — çünki istifadəçilər səhifəsi READ-ONLY-dir (yaratmaq/silmək YOXDUR), API-yə HEÇ NƏ GÖNDƏRİLMİR.

Hissə 9: Sabitlər

src/lib/constants/productTypes.js

```

export const PRODUCT_TYPE_LABELS = {
  kg: 'Kiloqram',
  gr: 'Qram',
  litre: 'Litr',
  ml: 'Millilitr',
  meter: 'Metr',
  cm: 'Santimetr',
  mm: 'Millimetr',
  piece: 'Ədəd',
  packet: 'Paket',
  box: 'Qutu',
}

export const PRODUCT_TYPE_OPTIONS = Object.keys(PRODUCT_TYPE_LABELS)

const WEIGHT_BASED_TYPES = ['kg', 'gr', 'litre', 'ml']

export const productTypeBadgeColor = (type) => (WEIGHT_BASED_TYPES.includes(type) ? 'purple' : 'green')

```

- `PRODUCT_TYPE_LABELS` — API-nin ENUM DƏYƏRLƏRİNİ ("kg" , "piece" və s.) Azərbaycanca ETİKETLƏRƏ ("Kiloqram" , "Ədəd") BAĞLAYAN xəritə (obyekt).
- `Object.keys(PRODUCT_TYPE_LABELS)` — obyektin BÜTÜN AÇARLARINI (['kg', 'gr', 'litre', ...]) MASSİV kimi qaytarır — bu, FORMDAKI `<select>` -in `<option>` -lərini YARATMAQ üçün istifadə OLUNUR.
- `WEIGHT_BASED_TYPES.includes(type)` — `.includes(...)` bir massivin MÜƏYYƏN dəyəri EHTİVA edib-etmədiyini yoxlayır (`true` / `false`).
- `productTypeBadgeColor` — BİR OX FUNKSIYASI, `type` PARAMETRİNƏ GÖRƏ BADGE RƏNGİNİ ("purple" — çəki əsaslı ölçülər üçün, ya "green" — say əsaslı ölçülər üçün) QAYTARIR.

src/lib/constants/orderStatus.js

```
export const ORDER_STATUS_LABELS = {
  PENDING: 'Gözləyir',
  CONFIRMED: 'Təsdiqləndi',
  PREPARING: 'Hazırlanır',
  READY: 'Hazırdır',
  DELIVERED: 'Çatdırıldı',
  CANCELLED: 'Ləğv edildi',
}

export const ORDER_STATUS_BADGE_COLOR = {
  PENDING: 'amber',
  CONFIRMED: 'blue',
  PREPARING: 'purple',
  READY: 'blue',
  DELIVERED: 'green',
  CANCELLED: 'red',
}

export const ORDER_STATUS_OPTIONS = Object.keys(ORDER_STATUS_LABELS)
```

YUXARIDAKI ilə EYNİ MƏNTİQ, sadəcə sifariş STATUSLARI üçün — ETİKETLƏR, BADGE RƏNGLƏRİ, VƏ dropdown seçimləri.

Hissə 10: formatDate.js

```
export function formatDate(isoString) {
  if (!isoString) return ''
  const d = new Date(isoString)
  return `${String(d.getDate()).padStart(2, '0')}.${String(d.getMonth() + 1).padStart(2, '0')}.${String(d.getFullYear())}`
}
```

- `if (!isoString) return ''` — TARİX VERİLMƏYİBSƏ (`null` / `undefined` /boş), BOŞ STRING QAYTAR (aşağıdakı sətirlər İŞLƏMƏSİN, xəta versin deyə).
- `new Date(isoString)` — API-DƏN GƏLƏN ISO string-i (`"2025-06-12T05:37:56.753Z"`) JavaScript-in daxili `Date` OBYEKTİNƏ çevirir.
- `d.getDate()` — AYIN günü (1-31), `d.getMonth()` — AY (DİQQƏT: 0-DAN başlayır! Yanvar=0, Dekabr=11 — ONA GÖRƏ + 1 ƏLAVƏ OLUNUB), `d.getFullYear()` — İL (4 rəqəmli).
- `String(...).padStart(2, '0')` — RƏQƏMİ STRINGƏ çevirir, sonra `padStart(2, '0')` ilə SOLDAN "0" ilə DOLDURUR ki, HƏMİŞƏ 2 RƏQƏM olsun (`5` → `"05"` , `12` → `"12"`).
- Nəticə: `"12.06.2025"` formatında bir STRING.

Hissə 11: TanStack Query

`src/lib/queryClient.js`

TanStack Query nə üçündür? Serverdən data ÇƏKMƏK (fetch), onu YADDAŞDA (cache) SAXLAMAQ, KÖHNƏLƏNDƏ YENİLƏMƏK, YÜKLƏNMƏ/XƏTA vəziyyətlərini İDARƏ ETMƏK — bunların HAMISINI ƏL İLƏ

(`useState` + `useEffect` ilə) yazmaq ƏVƏZİNƏ, bu kitabxana HAZIR HƏLL TƏQDİM EDİR.

```
import { QueryClient, QueryCache, MutationCache } from '@tanstack/react-query'
import { toast } from 'sonner'

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 15_000 },
  },
  queryCache: new QueryCache({
    onError: (err) => toast.error(err.message),
  }),
  mutationCache: new MutationCache({
    onError: (err) => toast.error(err.message),
  }),
})
```

Sətir-sətir:

- `new QueryClient({...})` — `new` ACHAR SÖZÜ ilə BİR "SİNİF"DƏN (class) YENİ BİR OBYEKT (instans) YARADILIR — `QueryClient` BÜTÜN CACHE-i VƏ AYARLARI SAXLAYAN "BEYİNDİR".
- `defaultOptions.queries.staleTime: 15_000` — `15_000` = 15000 (JavaScript-də ƏDƏDLƏRİN İÇİNDƏ `_` OXUNUŞU ASANLAŞDIRMAQ ÜÇÜN İŞLƏDİLİR, HEÇ BİR RİYAZİ MƏNASI YOXDUR — sadəcə 15000 MİLLİSANIYƏ = 15 SANİYƏ). `staleTime` — BİR DATA ÇƏKİLDİKDƏN SONRA NƏ QƏDƏR MÜDDƏT "TƏZƏ" (fresh) SAYILSIN. 15 SANİYƏ ƏRZİNDƏ EYNİ SƏHİFƏYƏ GERİ QAYITSANIZ, YENİDƏN SORĞU GETMİR, CACHE-DƏKİ DATA GÖSTƏRİLİR.
- `queryCache: new QueryCache({ onError: ... })` — BÜTÜN `useQuery` (data OXUMA) sorğuları ÜÇÜN QLOBAL BİR XƏTA-TUTUCU. İSTƏNİLƏN SƏHİFƏDƏ İSTƏNİLƏN `useQuery` XƏTA VERSƏ, BU FUNKSIYA İŞƏ DÜŞÜR, `toast.error(...)` İLƏ BİLDİRİŞ GÖSTƏRİR — HƏR SƏHİFƏDƏ AYRI-AYRI XƏTA İDARƏ ETMƏYƏ EHTİYAC QALMIR.
- `mutationCache: new MutationCache({ onError: ... })` — EYNİ MƏNTİQ, AMMA `useMutation` (data YAZMA — yaratma/yeniləmə/silmə) ÜÇÜN.

Bu `queryClient` obyektini `App.jsx` -də `<QueryClientProvider client={queryClient}>` İLƏ TƏTBİQƏ "TANIDILIR" (yuxarıda görmüşdük).

Hissə 12: Shared komponentlər

Bu bölmədəki HAMISI `src/shared/` qovluğundadır — TƏTBİQİN İSTƏNİLƏN YERİNDƏ TƏKRAR İSTİFADƏ OLUNAN, "AĞILLI" (data ilə işləməyən, sadəcə görünüş) komponentlərdir.

Button.jsx

```
import styles from './Button.module.css'

export default function Button({
  variant = 'solid',
  icon: Icon,
  iconSize = 16,
  fullWidth = false,
```

```

    block = false,
    className = '',
    children,
    type = 'button',
    ...rest
  }) {
    const classes = [
      styles.btn,
      styles[variant],
      fullWidth ? styles.fullWidth : '',
      block ? styles.block : '',
      className,
    ]
    .filter(Boolean)
    .join(' ')

    return (
      <button type={type} className={classes} {...rest}>
        {Icon && <Icon size={iconSize} />}
        {children}
      </button>
    )
  }
}

```

- `variant = 'solid'` — DESTRUCTURING zamanı = İLƏ **DEFAULT DƏYƏR** TƏYİN OLUNUR: `<Button>` İSTİFADƏ EDƏNDƏ `variant` PROP-UNU VERMƏSƏNİZ, AVTOMATİK `'solid'` OLUR.
- `icon: Icon` — DESTRUCTURING İLƏ EYNİ ZAMANDA **YENİDƏN ADLANDIRMA**: PROP-UN ADI `icon` - DUR, AMMA BİZ ONU YEREL DƏYİŞƏN KİMİ `Icon` (BÖYÜK HƏRFLƏ) ADLANDIRIRIQ — ÇÜNKİ JSX-DƏ `<Icon/>` YAZMAQ ÜÇÜN DƏYİŞƏN BÖYÜK HƏRFLƏ BAŞLAMALIDIR (React KİÇİK HƏRFLƏ BAŞLAYANLARI ADI HTML TEQİ SANIR).
- `...rest` — YUXARIDA İZAH OLUNAN "REST" — `variant`, `icon`, `iconSize` VƏ S. ÇIXARILDIQDAN SONRA, QALAN BÜTÜN PROP-LAR (`onClick`, `disabled` VƏ S.) `rest` OBYEKTİNƏ YIĞILIR.
- `classes = [...].filter(Boolean).join(' ')` — BİR NEÇƏ CSS KLASINI ŞƏRTİ OLARAQ BİRLƏŞDİRMƏK ÜÇÜN ÜMUMİ TRİK: MASSİV QURULUR (BƏZİ ELEMENTLƏR BOŞ STRİNG `' '` OLA BİLƏR), `.filter(Boolean)` BOŞ STRİNGLƏRİ (VƏ `false` / `null` / `undefined` -U) MASSİVDƏN ÇIXARIR (`Boolean` FUNKSİYASI HƏR ELEMENTİ `true` / `false` -A ÇEVİRİR, `filter` YALNIZ `true` OLANLARI SAXLAYIR — BOŞ STRİNG `Boolean('') → false` OLDUĞU ÜÇÜN ATILIR), `.join(' ')` QALANLARI BOŞLUQLA BİRLƏŞDİRİB TƏK STRİNG EDİR.
- `<button type={type} className={classes} {...rest}>` — `{...rest}` BURADA DA SPREAD-DİR, AMMA JSX DAXİLİNDƏ: `rest` OBYEKTİNDƏKİ HƏR AÇAR-DƏYƏRİ BU ELEMENTƏ AYRI-AYRI PROP KİMİ "TÖKÜR" (`onClick={rest.onClick}`, `disabled={rest.disabled}` YAZMAĞA BƏRABƏRDİR, SADƏCƏ QISA).
- `{Icon && <Icon size={iconSize} />}` — `Icon` PROP-U VERİLİBSƏ (İKON KOMPOENTİDİRSƏ), ONU RENDER ET, VERİLMƏYİBSƏ (`undefined` / `false` -DURSA) HEÇ NƏ GÖSTƏRMƏ.

Modal.jsx

```

import { useEffect, useRef } from 'react'
import { X } from 'lucide-react'
import styles from './Modal.module.css'

```



```

export default function Modal({ open, onClose, title, children, wide = false }) {
  const closeBtnRef = useRef(null)

  useEffect(() => {
    if (!open) return

    closeBtnRef.current?.focus()

    const handleKeyDown = (e) => {
      if (e.key === 'Escape') onClose()
    }
    document.addEventListener('keydown', handleKeyDown)
    return () => document.removeEventListener('keydown', handleKeyDown)
  }, [open, onClose])

  if (!open) return null

  return (
    <div className={styles.overlay}>
      <div className={` ${styles.card} ${wide ? styles.cardWide : ''}` role="dialog" aria-
modal="true" aria-label={title}>
        <div className={styles.header}>
          {title ? <h3 className={styles.title}>{title}</h3> : <div />}
          <button type="button" ref={closeBtnRef} onClick={onClose} className=
{styles.closeBtn} aria-label="Bağla">
            <X size={20} />
          </button>
        </div>
        {children}
      </div>
    </div>
  )
}

```

- `useRef(null)` — `useState` -ə OXŞAYIR, AMMA DƏYƏR DƏYİŞƏNDƏ KOMPOONENTİ YENİDƏN RENDER ETMİR. ADƏTƏN BİR DOM ELEMENTİNƏ (BURADA — BAĞLAMA DÜYMƏSİNƏ) BİRBAŞA MÜRACİƏT ETMƏK ÜÇÜN İŞLƏDİLİR. `ref={closeBtnRef}` DÜYMƏYƏ VERİLƏNDƏ, `closeBtnRef.current` O DÜYMƏNİN ƏSL DOM ELEMENTİNƏ "İŞARƏ EDİR".
- `useEffect(() => {...}, [open, onClose])` — `open` VƏ YA `onClose` DƏYİŞƏNDƏ İŞƏ DÜŞÜR.
 - `if (!open) return` — MODAL BAĞLIDIRSA, HEÇ NƏ ETMƏ (ERKƏN ÇIXIŞ).
 - `closeBtnRef.current?.focus()` — MODAL AÇILANDA, BAĞLAMA DÜYMƏSİNƏ AVTOMATİK "FOCUS" (KLAVIATURA DİQQƏTİ) VERİR — BU, ACCESSIBILITY (ƏLÇATANLIQ) ÜÇÜNDÜR: KLAVIATURA İLƏ İDARƏ EDƏN İSTİFADƏÇİ MODAL AÇILAN KİMİ NƏ HARADA OLDUĞUNU BİLİR.
 - `handleKeyDown` FUNKSİYASI — HƏR DÜYMƏ BASILANDA İŞƏ DÜŞÜR, `Escape` BASILIBSA `onClose()` ÇAĞIRIR.
 - `document.addEventListener('keydown', handleKeyDown)` — BÜTÜN SƏNƏDƏ (SƏHİFƏYƏ) BU DİNLƏYİCİNİ ƏLAVƏ EDİR.
 - `return () => document.removeEventListener(...)` — BU, **CLEANUP FUNKSİYASIDIR** — `useEffect` -İN DAXİLİNDƏN `return` EDİLƏN FUNKSİYA, EFEKT YENİDƏN İŞƏ DÜŞMƏZDƏN

ƏVVƏL (VƏ YA KOMPONENT YOX OLANDA) ÇAĞIRILIR. BURADA: DİNLƏYİCİNİ SİLİR Kİ, MODAL BAĞLANDIQDAN SONRA DA `Escape` DİNLƏNMƏSİN (YADDAŞ SIZMASININ QARŞISINI ALIR).

- `if (!open) return null` — MODAL BAĞLIDIRSA, HEÇ NƏ RENDER ETMİR (React-Də `null` QAYTARMAQ "BURADA HEÇ NƏ GÖSTƏRMƏ" DEMƏKDİR).
- `role="dialog" aria-modal="true"` — SCREEN READER (EKRAN OXUYUCU, KOR İSTİFADƏÇİLƏR ÜÇÜN) PROQRAMLARINA "BU BİR DİALOQ PƏNCƏRƏSİDİR" DEMƏK ÜÇÜN XÜSUSİ HTML ATRİBUTLARI.
- `{title ? <h3>...</h3> : <div />}` — BAŞLIQ VERİLİBSƏ GÖSTƏR, VERİLMƏYİBSƏ BOŞ BİR `<div/>` QOY (YERLƏŞMƏ POZULMASIN DEYƏ — CSS-Də `justify-content: space-between` KİMİ BİR ŞEY OLDUĞU ÜÇÜN, BAĞLAMA DÜYMƏSİ HƏMİŞƏ SAĞDA QALSIN).
- `{children}` — `<Modal>...BURADA...</Modal>` YAZILANDA, İÇİNDƏKİ HƏR ŞEY `children` PROP-U KİMİ GƏLİR VƏ BURADA RENDER OLUNUR.

ConfirmModal.jsx

`Modal.jsx` İLƏ EYNİ MƏNTİQ (ESCAPE, FOCUS), AMMA AYRICA, DAHA SADƏ BİR KOMPONENTDİR — "ƏMİNSİNİZMİ?" TIPLI SUALLAR ÜÇÜN. FƏRQİ: `role="alertdialog"` (ADİ DİALOQ DEYİL, XƏBƏRDARLIQ DİALOQU) VƏ `Button ref={cancelBtnRef}` — BURADA REACT 19-UN "ref PROP KİMİ" XÜSUSİYYƏTİ İŞLƏDİLİR: `Button` ADİ FUNKSIYA KOMPONENTİDİR (forwardRef-SİZ), AMMA REACT 19-DA `ref` -İ BİLƏVASİTƏ QƏBUL EDƏ BİLİR VƏ `...rest` -İN İÇİNDƏ `<button>` -Ə ÖTÜRÜLÜR.

Badge.jsx

```
import styles from './Badge.module.css'

export default function Badge({ color = 'green', children }) {
  return <span className={` ${styles.badge} ${styles[color]} `}>{children}</span>
}
```

`styles[color]` — DİNAMİK KLAS SEÇİMİ: `color` PROP-U "green", "blue", "amber" VƏ S. OLA BİLƏR, `styles[color]` HƏMİN ADLI CSS KLASINI OXUYUR (`styles.green`, `styles.blue` KİMİ — SADƏCƏ DƏYİŞƏN İLƏ). MİSAL: `<Badge color="red">Ləğv edildi</Badge>` — QIRMIZI FONLU BADGE.

StatCard.jsx

```
export default function StatCard({ label, value, icon: Icon, color }) {
  return (
    <div className={styles.card}>
      <span className={styles.label}>{label}</span>
      <span className={styles.value}>
        <Icon size={16} color={color} />
        {value}
      </span>
    </div>
  )
}
```

SADƏ BİR "KART" — ETİKET (`label`), RƏQƏM (`value`) VƏ RƏNGLİ İKON GÖSTƏRİR. ORDERS SƏHİFƏSİNDƏKİ 6 STATİSTİKA KARTI BUNDAN İSTİFADƏ EDİR.

ActionMenu.jsx (ən MÜRƏKKƏB SHARED KOMPONENT)

```
import { useEffect, useRef, useState } from 'react'
import { createPortal } from 'react-dom'
import { MoreVertical, Eye, Pencil, Trash2 } from 'lucide-react'
import styles from './ActionMenu.module.css'

export default function ActionMenu({ onView, onEdit, onDelete }) {
  const [open, setOpen] = useState(false)
  const [pos, setPos] = useState({ top: 0, left: 0 })
  const triggerRef = useRef(null)
  const menuRef = useRef(null)

  const openMenu = () => {
    const rect = triggerRef.current?.getBoundingClientRect()
    const menuWidth = 150
    setPos({
      top: rect?.bottom + 4,
      left: Math.max(8, rect?.right - menuWidth),
    })
    setOpen(true)
  }

  useEffect(() => {
    if (!open) return undefined

    const handlePointerDown = (e) => {
      if (triggerRef.current?.contains(e.target) || menuRef.current?.contains(e.target))
        return
      setOpen(false)
    }
    const handleScroll = () => setOpen(false)

    document.addEventListener('mousedown', handlePointerDown)
    window.addEventListener('scroll', handleScroll, true)
    window.addEventListener('resize', handleScroll)

    return () => {
      document.removeEventListener('mousedown', handlePointerDown)
      window.removeEventListener('scroll', handleScroll, true)
      window.removeEventListener('resize', handleScroll)
    }
  }, [open])

  const handleSelect = (fn) => {
    setOpen(false)
    fn?.()
  }

  return (
    <>
```

```

    <button type="button" ref={triggerRef} className={...} onClick={() => (open ?
setOpen(false) : openMenu())} aria-label="Əməliyyatlar">
    <MoreVertical size={18} />
  </button>
  {open &&
    createPortal(
      <div ref={menuRef} className={styles.menu} style={{ top: pos.top, left: pos.left
}}>
        <button onClick={() => handleSelect(onView)}><Eye size={14} /> Bax</button>
        <button onClick={() => handleSelect(onEdit)}><Pencil size={14} />
Düzəlt</button>
        <button onClick={() => handleSelect(onDelete)}><Trash2 size={14} /> Sil</button>
      </div>,
      document.body,
    )}
  </>
)
}

```

Bu, CƏDVƏLLƏRDƏKİ "ÜÇ NÖQTƏ" (:) MENYUSUDUR (Bax/Düzəlt/Sil).

- `useState({ top: 0, left: 0 })` — MENYUNUN EKRANDA HANSI KOORDİNATDA GÖSTƏRİLƏCƏYİNİ SAXLAYIR (PİKSELLƏRLƏ).
- `triggerRef` / `menuRef` — İKİ AYRI `useRef` — BİRİ ":" DÜYMƏSİNƏ, DİGƏRİ AÇILAN MENYUNUN ÖZÜNƏ İŞARƏ EDİR.
- `openMenu()` :
 - `triggerRef.current.getBoundingClientRect()` — DÜYMƏNİN EKRANDAKI DƏQİQ MÖVQEYİNİ (YUXARI/AŞAĞI/SOL/SAĞ KƏNARLARI) QAYTARAN BROWSER FUNKSİYASI.
 - `top: rect.bottom + 4` — MENYU DÜYMƏNİN 4px AŞAĞISINDA AÇILSIN.
 - `left: Math.max(8, rect.right - menuWidth)` — MENYU DÜYMƏNİN SAĞ KƏNARINA HİZALANSIN (SAĞDAN SOLA DOĞRU AÇILSIN), AMMA `Math.max(8, ...)` İLƏ EKRANIN SOL KƏNARINDAN ƏN AZI 8px UZAQ QALSIN (EKRANDAN "ÇIXMASIN").
- `useEffect` (MENYU AÇIQ OLANDA) — İKİ HADİSƏYƏ QULAQ ASIR:
 - `handlePointerDown` — SƏHİFƏNİN İSTƏNİLƏN YERİNƏ KLİKLƏNƏNDƏ İŞƏ DÜŞÜR. `if (...contains(e.target)) return` — ƏGƏR KLİK EDİLƏN YER DÜYMƏNİN VƏ YA MENYUNUN ÖZÜNÜN İÇİNDƏDİRSƏ, HEÇ NƏ ETMƏ (BAĞLAMA); ƏKS HALDA (MENYUDAN KƏNARA KLİKLƏNİBSƏ) MENYUNU BAĞLA.
 - `handleScroll` — SƏHİFƏ SÜRÜŞDÜRÜLƏNDƏ VƏ YA PƏNCƏRƏ ÖLÇÜSÜ DƏYİŞƏNDƏ MENYUNU BAĞLAYIR (ÇÜNKİ KOORDİNATLAR ARTIQ SƏHV OLARDI).
 - `true` (ÜÇÜNCÜ ARQUMENT `addEventListener` -DƏ) — "CAPTURE PHASE" Dİ İSTƏYİR, YƏNİ HƏTTA SƏHİFƏNİN İSTƏNİLƏN YERİNDƏ (İÇ-İÇƏ SCROLL KONTEYNERLƏRDƏ BELƏ) SCROLL OLARSA TUTULSUN.
- `createPortal(...)` **NƏ ÜÇÜNDÜR?** NORMAL HALDA, JSX ELEMENTLƏRİ ÖZ "VALIDEYN" ELEMENTİNİN İÇİNDƏ RENDER OLUNUR. AMMA BU MENYU BİR CƏDVƏL XANASININ (`<td>`) İÇİNDƏDİR, CƏDVƏL İSƏ `overflow-x: auto` (ÜFÜQİ SÜRÜŞDÜRMƏ) OLAN BİR QUTUYA BÜRÜNÜB — ƏGƏR MENYU O QUTUNUN İÇİNDƏ QALSAYDI, "KƏSİLƏRDİ" (GÖRÜNMƏZ OLARDI, KONTEYNERDƏN DIŞARI ÇIXA BİLMƏZDİ). `createPortal(jsxElementi, document.body)` — BU JSX-İ, REACT AĞACINDA HARADA OLMASINDAN ASILI OLMAYARAQ, DOM-DA BİRBAŞA `<body>` -NİN İÇİNƏ "İŞİN" EDİR (TELEPORT EDİR KİMİ) — BELƏLİKLƏ HEÇ BİR KONTEYNER TƏRƏFİNDƏN KƏSİLMİR.

- `handleSelect(fn)` — SEÇİM EDİLƏNDƏ (BAX/DÜZƏLT/SİL) ƏVVƏLCƏ MENYUNU BAĞLAYIR, SONRA `fn?.()` İLƏ VERİLƏN FUNKSİYANI (`onView` / `onEdit` / `onDelete`) ÇAĞIRIR (`?.()` — FUNKSİYA VERİLMƏYİBSƏ XƏTA VERMƏDƏN KEÇİR).
- `<>...</>` — **FRAGMENT**DIR. REACT-DƏ BİR KOMPONENT YALNIZ BİR "KÖK" ELEMENT QAYTARA BİLƏR — AMMA BURADA HƏM DÜYMƏ, HƏM DƏ (ŞƏRTİ) MENYU EYNİ SƏVİYYƏDƏ OLMALIDIR. ƏLAVƏ, LAZIMSIZ BİR `<div>` YARATMAMAQ ÜÇÜN, "GÖRÜNMƏZ" BİR SARGI KİMİ `<>...</>` (FRAGMENT) İŞLƏDİLİR.

Table.jsx

```
export function Table({ columns, minWidth = 720, children }) {
  return (
    <div className={styles.scroll}>
      <table className={styles.table} style={{ minWidth }}>
        <thead>
          <tr>
            {columns.map((col) => (
              <th key={col.key} style={{ width: col.width }}>{col.label}</th>
            ))}
          </tr>
        </thead>
        <tbody>{children}</tbody>
      </table>
    </div>
  )
}

export function TableEmptyRow({ colSpan, children = 'Nəticə tapılmadı' }) {
  return (
    <tr className={styles.emptyRow}>
      <td colSpan={colSpan}>{children}</td>
    </tr>
  )
}

export default Table
```

BU FAYL **İKİ NAMED EXPORT** (`Table` , `TableEmptyRow`) VƏ BİR **DEFAULT EXPORT** (`Table` -İN ÖZÜ) VERİR — İKİ CÜR DƏ IMPORT OLUNA BİLƏR: `import Table from '...'` (default) YA DA `import { Table, TableEmptyRow } from '...'` (named). SƏHİFƏLƏR HƏR İKİSİNİ EYNİ VAXTDA `{ Table, TableEmptyRow }` ŞƏKLİNDƏ İDXAL EDİR.

`Table` KOMPONENTİ ÖZÜ DATA "BİLMİR" — SƏHİFƏLƏR ONA `columns` (SÜTUN TƏSVİRLƏRİ) VƏ `children` (CƏDVƏLİN SƏTİRLƏRİ, YƏNİ `<tr>...</tr>` ELEMENTLƏRİ) VERİR, O SADƏCƏ `<thead>` -İ `columns` -DAN AVTOMATİK QURUR, `<tbody>` -NİN İÇİNƏ İSƏ `children` -İ (SƏHİFƏNİN ÖZÜNÜN YARATDIĞI SƏTİRLƏRİ) QOYUR.

`TableEmptyRow` — SİYAHİ BOŞ OLANDA ("Nəticə tapılmadı") GÖSTƏRİLƏN XÜSUSİ SƏTİR. `children = 'Nəticə tapılmadı'` — DEFAULT MƏTN, İSTƏSƏNİZ FƏRQLİ MƏTN VERƏ BİLƏRSİNİZ.

Loading.jsx

```
export default function Loading({ fullScreen = false }) {
  return (
    <div className={` ${styles.wrap} ${fullScreen ? styles.fullScreen : ''}`>
      <span className={styles.spinner} />
      <span className={styles.text}>Yüklənir...</span>
    </div>
  )
}
```

SPINNER (FIRLANAN DAİRƏ) VƏ MƏTN — CSS-DƏ (`Loading.module.css`) `border` XASSƏSİNDƏN İSTİFADƏ EDİLİB (4 FƏRQLİ RƏNGLİ KƏNAR + FIRLANMA ANİMASİYASI). `fullScreen` PROP-U İKİ FƏRQLİ REJİM ÜÇÜNDÜR: `false` (DEFAULT) — SƏHİFƏNİN İÇİNDƏ KİÇİK, YAN-YANA GÖRÜNÜŞ; `true` — BÜTÜN EKRANI ORTALAYIB TUTAN GÖRÜNÜŞ (ROUTE DƏYİŞƏNDƏ `Suspense` FALLBACK-I KİMİ İSTİFADƏ OLUNUR).

Thumbnail.jsx

```
export default function Thumbnail({ imageUrl, image, color, size = 'sm' }) {
  return (
    <span className={` ${styles.thumb} ${styles[size]}`} style={{ backgroundColor: color }}>
      {imageUrl ? <img src={imageUrl} alt="" className={styles.img} /> : image}
    </span>
  )
}
```

`imageUrl ? <img .../> : image` — ƏSL ŞƏKİL ÜNVANI (`imageUrl`) VARSA `<img>` TEQİ İLƏ GÖSTƏRİR, YOXDURSA `image` (EMOJİ, MƏSƏLƏN "👤") MƏTN KİMİ GÖSTƏRİLİR. `style={{ backgroundColor: color }}` — DİNAMİK FON RƏNGİ (CSS FAYLINDA STATİK YAZILA BİLMƏZ, ÇÜNKİ HƏR ELEMENTİN ÖZ RƏNGİ VAR).

ErrorBoundary.jsx

```
import { Component } from 'react'
import { ServerCrash } from 'lucide-react'
import Button from '@shared/Button/Button'
import styles from './ErrorBoundary.module.css'

export default class ErrorBoundary extends Component {
  state = { hasError: false }

  static getDerivedStateFromError() {
    return { hasError: true }
  }

  componentDidCatch(error, info) {
    console.error(error, info)
  }

  render() {
    if (!this.state.hasError) return this.props.children
  }
}
```

```

return (
  <div className={styles.page}>
    <span className={styles.badge}><ServerCrash size={32} /></span>
    <h1 className={styles.title}>Nəsə səhv getdi</h1>
    <p className={styles.text}>Sorgunu yerinə yetirmək mümkün olmadı. Bir az sonra
yeni cəhd edin.</p>
    <Button onClick={() => (window.location.href = '/sifarisler')}>Ana səhifəyə
qayıt</Button>
  </div>
)
}
}

```

BU FAYL, LAYİHƏDƏ YEGANƏ "CLASS COMPONENT" DİR — BÜTÜN DİGƏR HƏR ŞEY FUNKSİYA KOMPONENTLƏRİDİR. SƏBƏBİ: REACT-DA "ERROR BOUNDARY" (XƏTA TUTUCUSU) YALNIZ CLASS KOMPONENT KİMİ YAZILA BİLƏR — BU, REACT-IN ÖZ QAYDASIDIR, HOOK-LARLA (FUNKSİYA KOMPONENTLƏRİ İLƏ) BUNU ETMƏK MÜMKÜN DEYİL.

- `class ErrorBoundary extends Component` — `Component` REACT-IN ÖZ BAZA SİNFİDİR, `extends` İLƏ ONDAN "MİRAS ALIRIQ" (BÜTÜN XÜSUSİYYƏTLƏRİNİ ALIRIQ, ÜSTÜNƏ ÖZ MƏNTİQİMİZİ ƏLAVƏ EDİRİK).
- `state = { hasError: false }` — CLASS KOMPONENTİN "state"i (`useState` -İN CLASS VERSİYASI) — BAŞLANGICDA XƏTA YOXDUR.
- `static getDerivedStateFromError()` — REACT-IN ÖZÜ ÇAĞIRDIĞI XÜSUSİ BİR METODDUR: BU KOMPONENTİN **İSTƏNİLƏN UŞAĞINDA** (`children` -İN İÇİNDƏ, İSTƏNİLƏN DƏRİNLİKDƏ) BİR JAVASCRIPT XƏTASI BAŞ VERƏNDƏ, REACT AVTOMATİK BUNU ÇAĞIRIR, QAYTARILAN OBYEKT (`{ hasError: true }`) YENİ `state` OLUR.
- `componentDidCatch(error, info)` — EYNİ ANDA İŞƏ DÜŞƏN DİGƏR BİR METOD, XƏTANI (VƏ ONUN "STACK" MƏLUMATINI) KONSOLA (`console.error`) YAZMAQ ÜÇÜN İSTİFADƏ OLUNUR (DEBUQ ÜÇÜN FAYDALIDIR).
- `render()` — CLASS KOMPONENTLƏRDƏ JSX QAYTARAN METODDUR (FUNKSİYA KOMPONENTLƏRDƏ BUNUN ƏVƏZİNƏ SADƏCƏ FUNKSİYANIN ÖZÜ JSX QAYTARIR).
 - `if (!this.state.hasError) return this.props.children` — XƏTA YOXDURSA, SADƏCƏ NORMAL UŞAQLARI (`children`) GÖSTƏR (YƏNİ `<ErrorBoundary>` -NİN İÇİNDƏKİ ƏSL TƏTBİQİ).
 - XƏTA VARSA, ONUN ƏVƏZİNƏ BU "FALLBACK" (EHTİYAT) EKRANI GÖSTƏRİR — İKON, MƏTN, VƏ "ANA SƏHİFƏYƏ QAYIT" DÜYMƏSİ.
 - `onClick={() => (window.location.href = '/sifarisler')}` — DİQQƏT: `useNavigate()` YOX, ÇÜNKİ HOOK-LAR (`useNavigate` DA BİR HOOK-DUR) YALNIZ FUNKSİYA KOMPONENTLƏRİNDƏ İŞLƏYİR, CLASS KOMPONENTDƏ YOX — ONA GÖRƏ SADƏ `window.location.href = '...'` (BROWSER-İN ÖZÜNÜN NAVİQASIYA ÜSULU, TAM SƏHİFƏ YENİLƏMƏSİ İLƏ) İŞLƏDİLİB.
 - `this.state.hasError / this.props.children` — CLASS KOMPONENTLƏRDƏ `state / props` -A HƏMİŞƏ `this`. İLƏ MÜRACİƏT OLUNUR (FUNKSİYA KOMPONENTLƏRİNDƏ BELƏ DEYİL — ORADA BİRBAŞA `props.children` YA DA DESTRUCTURING İŞLƏDİLİR).

Harada istifadə olunur? `main.jsx` -də `<ErrorBoundary><App></ErrorBoundary>` — BÜTÜN TƏTBİQİ ƏHATƏ EDİR. TƏTBİQİN İSTƏNİLƏN YERİNDƏ (İSTƏNİLƏN SƏHİFƏDƏ) GÖZLƏNİLMƏZ BİR JS XƏTASI BAŞ VERSƏ, EKRAN AĞAPPAQ BOŞ QALMAQ ƏVƏZİNƏ BU SƏLİQƏLİ EKRAN GÖRÜNÜR.

Hissə 13: Custom hooks

useTitle.js

```
import { useEffect } from 'react'

export function useTitle(title) {
  useEffect(() => {
    const previousTitle = document.title
    document.title = title ? `${title} · Tik Tak Admin` : 'Tik Tak Admin'

    return () => {
      document.title = previousTitle
    }
  }, [title])
}
```

`document.title` — BRAUZERİN TAB BAŞLIĞIDIR. `previousTitle` — DƏYİŞDİRMƏZDƏN ƏVVƏLKİ BAŞLIĞI YADDA SAXLAYIR. `return () => { document.title = previousTitle }` — CLEANUP FUNKSİYASI: KOMPONENT EKRANDAN GEDƏNDƏ (MƏS. BAŞQA SƏHİFƏYƏ KEÇƏNDƏ), TAB BAŞLIĞINI KÖHNƏ HALINA QAYTARIR. HƏR SƏHİFƏ (`Categories.jsx` , `Orders.jsx` VƏ S.) ÖZ ADI İLƏ `useTitle('Kateqoriyalar')` ÇAĞIRIR.

useDebounce.js

```
import { useEffect, useState } from 'react'

export function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value)

  useEffect(() => {
    const timeout = setTimeout(() => setDebouncedValue(value), delay)
    return () => clearTimeout(timeout)
  }, [value, delay])

  return debouncedValue
}
```

"Debounce" NƏ DEMƏKDİR? İSTİFADƏÇİ AXTARIŞ QUTUSUNA YAZANDA, HƏR HƏRFDƏ SERVERƏ SORĞU GETSƏ, ÇOX LAZIMSIZ SORĞU OLARDI. DEBOUNCE — "İSTİFADƏÇİ YAZMAĞI DAYANDIRDIQDAN MÜƏYYƏN MÜDDƏT (500ms) SONRA DAVRAN" DEMƏKDİR.

- `setTimeout(() => setDebouncedValue(value), delay)` — `delay` (500) MİLLİSƏNİYƏ SONRA, `debouncedValue` -Nİ CARI `value` -YA BƏRABƏRLƏŞDİRƏCƏK BİR "SAAT" QURUR.
- `return () => clearTimeout(timeout)` — CLEANUP: ƏGƏR `value` 500ms BİTMƏMİŞ YENƏ DƏYİŞSƏ (İSTİFADƏÇİ YENİDƏN YAZIRSA), ƏVVƏLKİ "SAAT" LƏĞV OLUNUR, YENİSİ BAŞLAYIR — BELƏLİKLƏ YALNIZ İSTİFADƏÇİ 500ms ƏRZİNDƏ HEÇ NƏ YAZMASA, `debouncedValue` YENİLƏNİR.
- İSTİFADƏ: `AdminLayout.jsx` -də `useDebounce(search, 500)` — AXTARIŞ MƏTNİ ANINDA STATE-Ə YAZILIR (INPUT SÜRƏTLİ HİSS OLUNSUN DEYƏ), AMMA SƏHİFƏLƏRƏ ÖTÜRÜLƏN "DEBOUNCED" DƏYƏR

YALNIZ 500ms SONRA YENİLƏNİR — FİLTRLƏMƏ (VƏ MÜMKÜN OLAN SERVER SORĞUSU) BU QƏDƏR TEZ-TEZ İŞƏ DÜŞMÜR.

usePagination.js

```
import { useState } from 'react'

export function usePagination(items, initialPageSize = 5) {
  const [page, setPage] = useState(1)
  const [pageSize, setPageSizeState] = useState(initialPageSize)

  const setPageSize = (size) => {
    setPageSizeState(size)
    setPage(1)
  }

  const paged = items.slice((page - 1) * pageSize, page * pageSize)

  return { page, setPage, pageSize, setPageSize, paged }
}
```

- `items` — TAM (FİLTRLƏNMİŞ) SİYAHİ, `initialPageSize` — BAŞLANGIÇ SƏHİFƏ ÖLÇÜSÜ (DEFAULT 5).
- `page / pageSize` — İKİ AYRI `useState`, CARI SƏHİFƏ NÖMRƏSİ VƏ SƏHİFƏ BAŞINA NEÇƏ ELEMENT GÖSTƏRİLƏCƏYİ.
- `setPageSize(size)` — **ÖZ FUNKSIYAMIZDIR** (`useState` -İN ÖZ SETTER-İ DEYİL, BİZ YAZMIŞIQQ) — SƏHİFƏ ÖLÇÜSÜNÜ DƏYİŞDİRİR VƏ EYNİ ZAMANDA `page` -İ 1-Ə QAYTARIR (MƏNTİQLİDİR: SƏHİFƏ ÖLÇÜSÜ 5-DƏN 20-YƏ DƏYİŞSƏ, "8-Cİ SƏHİFƏ" ARTIQQ MƏNASIZ OLA BİLƏR, ONA GÖRƏ HƏMİŞƏ 1-Ə QAYIDIR).
- `paged = items.slice((page - 1) * pageSize, page * pageSize)` — TAM SİYAHINI, CARI SƏHİFƏYƏ UYGUN HISSƏYƏ "KƏSİR". MİSAL: `page=2, pageSize=5` OLSA, `slice(5, 10)` — YƏNİ 6-Cİ ELEMENTDƏN 10-CU ELEMENTƏ QƏDƏR.
- SON SƏTİR — OBYEKT QAYTARIR, SƏHİFƏLƏR BUNU `const { page, setPage, pageSize, paged } = usePagination(filtered)` ŞƏKLİNDƏ DESTRUCTURE EDİB İSTİFADƏ EDİR.

useCrudModal.js

```
import { useState } from 'react'

export function useCrudModal(emptyForm, toForm) {
  const [formOpen, setFormOpen] = useState(false)
  const [editing, setEditing] = useState(null)
  const [form, setForm] = useState(emptyForm)
  const [deleteTarget, setDeleteTarget] = useState(null)
  const [viewTarget, setViewTarget] = useState(null)

  const openCreate = (overrides = {}) => {
    setEditing(null)
    setForm({ ...emptyForm, ...overrides })
    setFormOpen(true)
  }
}
```

```

const openEdit = (item) => {
  setEditing(item)
  setForm(toForm(item))
  setFormOpen(true)
}

return { formOpen, setFormOpen, editing, form, setForm, deleteTarget, setDeleteTarget,
viewTarget, setViewTarget, openCreate, openEdit }
}

```

BU HOOK, KATEQORİYALAR/KAMPANİYALAR/MƏHSULLAR SƏHİFƏLƏRİNİN **ÜÇÜNDƏ DƏ EYNI OLAN** "YARAT/DÜZƏLT/SİL/BAX" MODAL MƏNTİQİNİ BİR YERƏ YIGIR.

- 5 `useState` — `formOpen` (FORMA MODALI AÇIQDIRMI), `editing` (HANSI ELEMENT DÜZƏLDİLİR — `null` -DURSA "YENİ YARATMA" REJİMİDİR), `form` (FORMANIN CARİ DƏYƏRLƏRİ), `deleteTarget` (SİLİNMƏK İSTƏNƏN ELEMENT), `viewTarget` (BAXILAN ELEMENT).
- `openCreate(overrides = {})` — YENİ ELEMENT YARATMAQ ÜÇÜN FORMANI AÇIR: `editing` -i `null` EDİR, `form` -U `emptyForm` -A (BAŞLANGIC BOŞ DƏYƏRLƏR) QAYTARIR — AMMA `{ ...emptyForm, ...overrides }` İLƏ, ÇAĞIRAN TƏRƏF ƏLAVƏ SPESİFİK DƏYƏRLƏR (`overrides`) VERƏ BİLƏR (MƏSƏLƏN, PRODUCTS SƏHİFƏSİ `openCreate({ category_id: '...' })` ÇAĞIRIR KI, DEFAULT KATEQORIYA SEÇİLİ GƏLSİN).
- `openEdit(item)` — MÖVCUD BİR ELEMENTİ DÜZƏLTMƏK ÜÇÜN: `editing` -i O ELEMENTƏ QOYUR, `form` -U İSƏ `toForm(item)` (SƏHİFƏNİN ÖZÜNÜN VERDİYİ ÇEVİRMƏ FUNKSIYASI, MƏSƏLƏN `Categories.jsx` -DƏKİ `toForm`) İLƏ DOLDURUR.

Hissə 14: `Pagination.jsx`

`src/utils/Pagination/Pagination.jsx` — **KOMPONENTDİR** (YUXARIDAKI `usePagination` HOOK-U İLƏ QARIŞDIRMAYIN, İKİSİ FƏRQLİ ŞEYDİR — İZAHİ ÜÇÜN AŞAĞIYA BAXIN).

```

export default function Pagination({ page, pageSize, total, onPageChange, onPageSizeChange
}) {
  const totalPages = Math.max(1, Math.ceil(total / pageSize))
  const start = total === 0 ? 0 : (page - 1) * pageSize + 1
  const end = Math.min(total, page * pageSize)

  const pageNumbers = []
  for (let p = 1; p <= totalPages; p += 1) {
    if (p === 1 || p === totalPages || Math.abs(p - page) <= 1) {
      pageNumbers.push(p)
    } else if (pageNumbers[pageNumbers.length - 1] !== '...') {
      pageNumbers.push('...')
    }
  }

  return (
    <div className={styles.wrap}>
      <span>{start}-{end} / {total} nəticə</span>
      <div className={styles.pages}>

```

```

    <button disabled={page <= 1} onClick={() => onPageChange(page - 1)}><ChevronLeft
size={16} /></button>
    {pageNumbers.map((p, idx) =>
      p === '...' ? <span key={`dots-${idx}`}>...</span> : (
        <button key={p} onClick={() => onPageChange(p)} className={p === page ?
styles.pageBtnActive : ''}>{p}</button>
      ),
    )}
    <button disabled={page >= totalPages} onClick={() => onPageChange(page + 1)}>
<ChevronRight size={16} /></button>
  </div>
  {onPageSizeChange && (
    <select value={pageSize} onChange={(e) => onPageSizeChange(Number(e.target.value))}>
      <option value={5}>5 / page</option>
      <option value={10}>10 / page</option>
      <option value={20}>20 / page</option>
    </select>
  )}
</div>
)
}

```

- `totalPages = Math.max(1, Math.ceil(total / pageSize))` — `Math.ceil` YUXARIYA YUVARLAQLAŞDIRIR (23 ELEMENT, `PAGESIZE=5` → $23/5=4.6$ → 5 SƏHİFƏ). `Math.max(1, ...)` — HƏTTA `total=0` OLSA BELƏ, ƏN AZI 1 SƏHİFƏ GÖSTƏRİR (0 SƏHİFƏ MƏNASIZ OLARDI).
- `start / end` — "1-5 / 23 nəticə" KİMİ MƏTNİN RƏQƏMLƏRİNİ HESABLAYIR.
- `for (let p = 1; p <= totalPages; p += 1)` — KLASSİK "FOR DÖVRÜ": `p` 1-DƏN `totalPages` -A QƏDƏR, HƏR ADDIMDA 1 ARTARAQ TƏKRARLANIR.
- `if (p === 1 || p === totalPages || Math.abs(p - page) <= 1)` — HANSI SƏHİFƏ NÖMRƏLƏRİ GÖRSƏNSİN QƏRARI: BİRİNCİ SƏHİFƏ, SON SƏHİFƏ, VƏ CARİ SƏHİFƏNİN ± 1 ƏTRAFINDAKILAR (`Math.abs` — MÜTLƏQ DƏYƏR, MƏNFİ/MÜSBƏT FƏRQİ FƏRQ ETMİR). MƏSƏLƏN 10 SƏHİFƏDƏN, CARİ 5-DİRSƏ: 1, 4, 5, 6, 10 GÖSTƏRİLİR, ARADAKILAR "..." İLƏ GİZLƏDİLİR.
- `else if (pageNumbers[pageNumbers.length - 1] !== '...')` — ARTIQQ SONUNCU ƏLAVƏ EDİLƏN "..." DEYİLSƏ, BİR "..." ƏLAVƏ ET (Kİ, "1", "...", "...", "4" KİMİ TƏKRARLANMƏSİN, YALNIZ BİR "...").
- `onPageChange / onPageSizeChange` — BU KOMPONENTİN ÖZÜNÜN STATE-İ YOXDUR, HAMISI PROP (VƏ CALLBACK) — SƏHİFƏLƏR (VƏ YA `usePagination` HOOK-U ARACILIĞI İLƏ) BU DƏYƏRLƏRİ VERİR.

Pagination.jsx (komponent) VS usePagination.js (hook) — FƏRQ NƏDİR?

- `usePagination` — **MƏNTİQDİR**. `useState` İLƏ `page / pageSize` -İ SAXLAYIR, MASSİVİ `.slice()` EDİR. HEÇ BİR JSX/GÖRÜNÜŞ YOXDUR.
- `Pagination` — **GÖRÜNÜŞDÜR**. DÜYMƏLƏR, SƏHİFƏ NÖMRƏLƏRİ, "5/page" SEÇİCİSİ. ÖZ STATE-İ YOXDUR, HƏR ŞEYİ PROP KİMİ ALIR.

BİR SƏHİFƏ İKİSİNİ **BİRLİKDƏ** İŞLƏDİR: `usePagination` DATA KƏSİR, `Pagination` KOMPONENTİ İSƏ ONUN NƏTİCƏSİNİ (`page`, `pageSize`, `total`) GÖSTƏRİR VƏ KLİKLƏRİ (`onPageChange`) GERİ HOOK-A ÖTÜRÜR.

Hissə 15: Layout

`src/layouts/AdminLayout.jsx`

```

import { useState } from 'react'
import { Outlet } from 'react-router-dom'
import Sidebar from '@components/Sidebar/Sidebar'
import Header from '@components/Header/Header'
import { useDebounce } from '@shared/hooks/useDebounce'
import styles from './AdminLayout.module.css'

export default function AdminLayout() {
  const [search, setSearch] = useState('')
  const debouncedSearch = useDebounce(search, 500)

  return (
    <div className={styles.page}>
      <div className={styles.headerBar}>
        <div className={styles.headerInner}>
          <Header search={search} onSearchChange={setSearch} />
        </div>
      </div>
      <div className={styles.bodyBar}>
        <div className={styles.bodyInner}>
          <Sidebar />
          <main className={styles.main}>
            <Outlet context={{ search: debouncedSearch }} />
          </main>
        </div>
      </div>
    </div>
  )
}

```

BU, 5 QORUNAN SƏHİFƏNİN (`AppRoutes.jsx` -Ə YENİDƏN BAXIN) "ÇƏRÇİVƏSİDİR" — SIDEBAR + HEADER + AXTARIŞ MƏNTİQİ.

- `search` — INPUT-DAKI XAM (ANINDA YENİLƏNƏN) MƏTN.
- `debouncedSearch` — 500ms GECİKMƏLİ VERSİYA (YUXARIDA İZAH OLUNAN `useDebounce`).
- `<Header search={search} onSearchChange={setSearch} />` — HEADER-Ə XAM MƏTNİ VƏ ONU DƏYİŞMƏK FUNKSIYASINI ÖTÜRÜR (INPUT SÜRƏTLİ HİSS OLUNSUN DEYƏ).
- `<Outlet context={{ search: debouncedSearch }} />` — BURADA `Outlet` -Ə (YUXARIDA İZAH OLUNDU) `context` PROP-U VERİLİR — BU, REACT-ROUTER-DOM-UN XÜSUSİ BİR MEXANİZMİDİR: `AdminLayout` -UN İÇİNDƏ RENDER OLUNAN İSTƏNİLƏN SƏHİFƏ (`Orders` , `Categories` VƏ S.) `useOutletContext()` HOOK-U İLƏ BU `{ search: debouncedSearch }` OBYEKTİNİ OXUYA BİLİR — BELƏLİKLƏ AXTARIŞ MƏTNİ VALİDEYNDƏN (LAYOUT-DAN) UŞAQLARA (SƏHİFƏLƏRƏ) "PROP DRILLING" (ƏL-ƏL ÖTÜRMƏ) OLMADAN ÇATIR.

`src/components/Sidebar/Sidebar.jsx`

BU FAYL, NAV LİNKLƏRİ + LOGOUT DÜYMƏSİ + **HOVER PREFETCH** (SİÇAN LİNKİN ÜZƏRİNƏ GƏLƏNDƏ DATA ƏVVƏLCƏDƏN ÇƏKMƏK) MƏNTİQİNİ SAXLAYIR.

```

const PREFETCH = {
  '/sifarisler': (queryClient) => {

```

```

    queryClient.prefetchQuery({ queryKey: ['orders'], queryFn: () =>
listOrders().then((data) => data.map(mapOrderFromApi)) })
    queryClient.prefetchQuery({ queryKey: ['orderStats'], queryFn: getOrderStats })
  },
  // ... digər 4 route üçün oxşar
}

export default function Sidebar() {
  const navigate = useNavigate()
  const logout = useAuthStore((s) => s.logout)
  const queryClient = useQueryClient()

  return (
    <aside className={styles.aside}>
      <nav className={styles.nav}>
        {navItems.map(({ to, label, icon: Icon }) => (
          <NavLink
            key={to}
            to={to}
            onMouseEnter={() => PREFETCH[to]?.(queryClient)}
            className={({ isActive }) => `${styles.link} ${isActive ? styles.linkActive :
''}`}
          >
            <Icon size={18} />
            {label}
          </NavLink>
        ))}
      </nav>
      <button onClick={() => { logout(); navigate('/login') }} className={styles.logoutBtn}>
        <LogOut size={18} /> Çıxış
      </button>
    </aside>
  )
}

```

- **PREFETCH** — BİR OBYEKT, AÇARLARI ROUTE YOLLARI (/sifarisler VƏ S.), DƏYƏRLƏRİ İSƏ FUNKSİYALARDIR. HƏR FUNKSİYA `queryClient.prefetchQuery({...})` ÇAĞIRIR — BU, `useQuery` -YƏ ÇOX BƏNZƏYİR, AMMA KOMPONENTİN İÇİNDƏN DEYİL, İMPERATİV (BİRBAŞA) ÇAĞIRILIR VƏ NƏTİCƏNİ RENDER ETMİR, SADƏCƏ CACHE-Ə YAZIR.
- `queryKey` / `queryFn` — **EYNI DƏYƏRLƏR** HƏMİN SƏHİFƏNİN ÖZ `useQuery` ÇAĞIRIŞINDA DA İŞLƏDİLİR (BU, VACİBDİR — YUXARIDA "HİSSƏ 8"-DƏ İZAH OLUNAN BUG-A GÖRƏ).
- `{navItems.map(({ to, label, icon: Icon }) => ...)}` — `navItems` MASSİVİNİ GƏZİB, HƏR BİRİ ÜÇÜN BİR `<NavLink>` YARADIR.
- `onMouseEnter={() => PREFETCH[to]?.(queryClient)}` — SİÇAN LİNKİN ÜZƏRİNƏ GƏLƏNDƏ, **PREFETCH** OBYEKTİNDƏN O YOLA (`to`) UYGUN FUNKSİYANI TAPIB (`?.` İLƏ — TAPILMASA XƏTA VERMİR) ÇAĞIRIR.
- `className={({ isActive }) => ...}` — `NavLink` -İN XÜSUSİ BİR XÜSUSİYYƏTİDİR: `className` PROP-UNA STRİNG ƏVƏZİNƏ BİR FUNKSİYA VERİLƏ BİLƏR, BU FUNKSİYA `{ isActive }` (CARİ URL BU LİNKƏ UYGUNDURMU) ALIR VƏ UYGUN KLASI QAYTARIR — BELƏLİKLƏ CARİ SƏHİFƏNİN LİNKİ "AKTİV" GÖRÜNÜR.

- `onClick={() => { logout(); navigate('/login') }}` — ƏVVƏLCƏ `logout()` (ZUSTAND STORE-U TƏMİZLƏYİR), SONRA `navigate('/login')` (BROWSER-İ `/login` -Ə APARIR).

`src/components/Header/Header.jsx`

SADƏ BİR KOMPONENT — BAŞLIQ + AXTARIŞ İNPUTU. ÖZ STATE-İ YOXDUR (`search` / `onSearchChange` `AdminLayout` -DAN GƏLİR — "CONTROLLED INPUT" NÜMUNƏSİDİR: `value={search}` İLƏ İNPUTUN DƏYƏRİ TAM OLARAQ REACT STATE-İNDƏN İDARƏ OLUNUR, `onChange` HƏR HƏRFDƏ `onSearchChange` (YƏNİ `AdminLayout` -UN `setSearch` -İ) ÇAĞIRIR.

Hissə 16: Səhifələr

`src/pages/Login/Login.jsx`

TAM BİR SƏHİFƏ NÜMUNƏSİ OLARAQ, BUNU DA SƏTİR-SƏTİR İZAH EDİRİK.

```
export default function Login() {
  useTitle('Giriş')
  const [phone, setPhone] = useState('')
  const [password, setPassword] = useState('')
  const [showPassword, setShowPassword] = useState(false)
  const [loading, setLoading] = useState(false)
  const navigate = useNavigate()
  const login = useAuthStore((s) => s.login)

  const handleSubmit = async (e) => {
    e.preventDefault()
    if (!phone.trim() || !password.trim()) {
      toast.error('Telefon və parolu daxil edin')
      return
    }
    setLoading(true)
    try {
      await login(phone.trim(), password)
      navigate('/sifarisler', { replace: true })
    } catch (err) {
      toast.error(err.message)
    } finally {
      setLoading(false)
    }
  }

  return (
    // ... JSX (aşağıda ayrıca izah olunur)
  )
}
```

- `useTitle('Giriş')` — TAB BAŞLIĞINI "Giriş · Tik Tak Admin" EDİR.
- 4 `useState` — TELEFON, PAROL, "PAROLU GÖSTƏR" VƏZİYYƏTİ, VƏ "YÜKLƏNİR" VƏZİYYƏTİ.
- `handleSubmit = async (e) => {...}` — FORMA GÖNDƏRİLƏNDƏ (SUBMIT) İŞƏ DÜŞÜR.

- `e.preventDefault()` — HTML FORMALARININ DEFAULT DAVRANIŞI SƏHİFƏNİ TAM YENİDƏN YÜKLƏMƏKDİR — BU SƏTİR ONUN QARŞISINI ALIR (REACT ÖZÜ İDARƏ ETSİN DEYƏ).
- `if (!phone.trim() || !password.trim())` — `.trim()` STRINGİN ƏVVƏLİNDƏKİ/SONUNDAKI BOŞLUQLARI SİLİR (Kİ, YALNIZ BOŞLUQ YAZILMASI "DOLU" SAYILMASIN). BOŞDURSA, XƏTƏ TOAST-I GÖSTƏRİLİR VƏ `return` İLƏ FUNKSİYA BURADA DAYANIR (DAHA AŞAĞI GETMİR).
- `setLoading(true)` — DÜYMƏNİ DEAKTİV EDİB "Yoxlanılır..." YAZDIRIR.
- `await login(phone.trim(), password)` — `useAuthStore` -DAKİ `login` FUNKSİYASINI ÇAĞIRIR (YUXARIDA İZAH OLUNDU) VƏ NƏTİCƏNİ GÖZLƏYİR.
- UĞURLU OLSA: `navigate('/sifarisler', { replace: true })` — İSTİFADƏÇİNİ SİFARİŞLƏR SƏHİFƏSİNƏ APARIR.
- UĞURSUZ OLSA (`catch`): `toast.error(err.message)` — AXİOS İNTERCEPTOR-UN VERDİYİ AZƏRBAYCANCA XƏTƏ MESAJINI GÖSTƏRİR.
- `finally { setLoading(false) }` — İTƏR UĞURLU, İTƏR UĞURSUZ, DÜYMƏNİ YENİDƏN AKTİV EDİR.

JSX Hissəsi:

```
<div className={styles.inputWrap}>
  <Phone size={16} className={styles.leadingIcon} />
  <input type="tel" value={phone} onChange={(e) => setPhone(e.target.value)} className=
  {`${styles.input} ${styles.hasLeadingIcon}`} />
</div>
```

- `Phone` İKONU MÜTLƏQ MÖVQELƏNDİRMƏ (CSS-DƏ `position: absolute`) İLƏ INPUTUN SOLUNDA "ÜZƏRİNƏ" QOYULUR.
- `.hasLeadingIcon` KLASİ INPUTA `padding-left: 40px` ƏLAVƏ EDİR Kİ, YAZILAN MƏTN İKONUN ALTINA DÜŞMƏSİN.
- PAROL INPUTUNDA HƏM `Lock` (SOLDA), HƏM DƏ KLİKLƏNƏ BİLƏN `Eye / EyeOff` DÜYMƏSİ (SAĞDA) VAR:

```
<input type={showPassword ? 'text' : 'password'} ... />
<button onClick={() => setShowPassword((s) => !s)}>
  {showPassword ? <EyeOff size={16} /> : <Eye size={16} />}
</button>
```

- `type={showPassword ? 'text' : 'password'}` — `showPassword true` -DURSA INPUTUN TİPİ "text" OLUR (PAROL AÇIQ GÖRÜNÜR), `false` -DURSA "password" (NÖQTƏLƏRLƏ GİZLİDİR).
- `setShowPassword((s) => !s)` — **FUNKSİYA FORMASINDA STATE YENİLƏMƏ**: `setX(yeniDəyər)` ƏVƏZİNƏ `setX((köhnəDəyər) => yeniDəyər)` YAZMAQ OLAR — REACT ƏN SON DƏYƏRİ BU FUNKSİYAYA ÖTÜRÜR, SİZ ONU ƏSAS GÖTÜRÜB YENİSİNİ HESABLAYIRSINIZ. BURADA: `!s` — CARI DƏYƏRİN ƏKSİ (TRUE→FALSE, FALSE→TRUE) — YƏNİ "AÇIQ/GİZLİ" ARASINDA KEÇİD.

src/pages/NotFound/NotFound.jsx

ÇOX SADƏDİR — 404 İKONU, MƏTN, VƏ "ANA SƏHİFƏYƏ QAYIT" DÜYMƏSİ (`useNavigate()` İLƏ, ÇÜNKİ BU BİR FUNKSİYA KOMPONENTDİR, `ErrorBoundary` -DƏN FƏRQLİ OLARAQ HOOK İŞLƏDƏ BİLİR).

src/pages/protected/Categories/Categories.jsx — CƏMİYYƏTİN ƏSAS CRUD

NÜMUNƏSİ

BU SƏHİFƏ, "CRUD" (Create/Read/Update/Delete — Yarat/Oxu/Yenilə/Sil) NÜMUNƏSİNİN ƏN TİPİK MİSALIDIR — CAMPAIGNS VƏ PRODUCTS SƏHİFƏLƏRİ DƏ EYNİ QURULUŞU TƏKRARLAYIR (KİÇİK FƏRQLƏRLƏ).

```
const emptyForm = { image: '🍷', color: '#f3f4f6', imageUrl: '', name: '', description: '' }

const toForm = (item) => ({
  image: item.image, color: item.color, imageUrl: item.imageUrl || '',
  name: item.name, description: item.description,
})

const columns = [ /* cədvəl sütunları */ ]

export default function Categories() {
  useTitle('Kateqoriyalar')
  const { search } = useOutletContext()
  const queryClient = useQueryClient()

  const { data: categories = [], isLoading: loading } = useQuery({
    queryKey: ['categories'],
    queryFn: () => listCategories().then((data) => data.map(mapCategoryFromApi)),
  })
```

- `emptyForm` — FORMA "YARAT" REJİMİNDƏ AÇILANDA BAŞLANGIÇ (BOŞ) DƏYƏRLƏR.
- `toForm` — YUXARIDA İZAH OLUNDU (`useCrudModal` -A VERİLİR), BİR SİYAHİ ELEMENTİNİ FORMA STRUKTURUNA ÇEVİRİR.
- `const { search } = useOutletContext()` — `AdminLayout` -DAN GƏLƏN AXTARIŞ MƏTNİNİ (DEBOUNCED) OXUYUR (YUXARIDA İZAH OLUNDU).
- `const queryClient = useQueryClient()` — TANSTACK QUERY-NİN QLOBAL `queryClient` OBYEKTİNƏ (`App.jsx` -DƏ `<QueryClientProvider>` -Ə VERİLƏN EYNİ OBYEKT) MÜRACİƏT EDİR — CACHE-İ ƏL İLƏ "TƏZƏLƏMƏK" (`invalidateQueries`) ÜÇÜN LAZIMDIR.
- `useQuery({ queryKey: ['categories'], queryFn: ... })` — BU, TANSTACK QUERY-NİN ƏSAS HOOK-UDUR:
 - `queryKey: ['categories']` — CACHE-DƏ BU DATANI TANIYAN "ETİKET" (MASSİV FORMASINDA, ÇÜNKİ MÜRƏKKƏB AÇARLAR DA OLA BİLƏR, MƏS. `['product', 5]`).
 - `queryFn` — SORĞUNU FAKTİKİ EDƏN FUNKSİYA. `listCategories()` (SERVIS) ÇAĞIRILIR, NƏTİCƏ (XAM API MASSİVİ) `.then((data) => data.map(mapCategoryFromApi))` İLƏ HƏR ELEMENTİ UI FORMATINA ÇEVİRİR.
 - QAYTARDIĞI OBYEKTDƏN `data` (NƏTİCƏ) VƏ `isLoading` (HAZIRDA YÜKLƏNİRMİ) ÇIXARILIR. `data: categories = []` — DESTRUCTURING ZAMANI **HƏM YENİDƏN ADLANDIRMA** (`data` → `categories`), **HƏM DƏ DEFAULT DƏYƏR** (`= []`) EYNI ANDA: DATA HƏLƏ GƏLMƏYİBSƏ (`undefined` -DURSA), `categories` BOŞ MASSİV OLUR (Kİ, `.filter()` XƏTA VERMƏSİN).

```
const invalidate = () => queryClient.invalidateQueries({ queryKey: ['categories'] })
const createMutation = useMutation({
  mutationFn: createCategory,
  onSuccess: async () => {
    await invalidate()
    toast.success('Kateqoriya yaradıldı')
```



```
},  
})
```

- `invalidate` — KİÇİK BİR KÖMƏKÇİ FUNKSİYA, `['categories']` KEY-İNİ "KÖHNƏLMİŞ" (STALE) ELAN EDİR — TANSTACK QUERY AVTOMATİK YENİDƏN SORĞU GÖNDƏRİR VƏ SİYAHİ TƏZƏLƏNİR.
- `useMutation({...})` — `useQuery` -DƏN FƏRQLİ OLARAQ, DATA **YAZMAQ** (DƏYİŞDİRMƏK) ÜÇÜNDÜR, AVTOMATİK İŞƏ DÜŞMÜR — SİZ ONU ƏL İLƏ ÇAĞIRMALISINIZ (AŞAĞIDA `mutateAsync(...)`).
 - `mutationFn: createCategory` — HANSI SERVİS FUNKSİYASI ÇAĞIRILACAQ.
 - `onSuccess: async () => {...}` — SORĞU UĞURLA BİTƏNDƏ İŞƏ DÜŞÜR: ƏVVƏLCƏ `invalidate()` (SİYAHINI YENİLƏ), SONRA `toast.success(...)` (BİLDİRİŞ GÖSTƏR). **XƏTA** HAL ÜÇÜN AYRICA HEÇ NƏ YAZILMAYIB — ÇÜNKİ `queryClient.js` -DƏKİ QLOBAL `mutationCache.onError` HAMISI ÜÇÜN AVTOMATİK İŞLƏYİR (HİSSƏ 11-Ə BAXIN).
- EYNİ MODELLE `updateMutation` VƏ `deleteMutation` DA VAR (KODDA GÖRÜRSÜNÜZ).

```
const filtered = useMemo(  
  () => categories.filter((c) => `${c.name} ${c.description}`  
    .toLowerCase().includes(search.toLowerCase())),  
  [categories, search],  
)  
const { page, setPage, pageSize, paged } = usePagination(filtered)  
  
const { formOpen, setFormOpen, editing, form, setForm, deleteTarget, setDeleteTarget,  
  viewTarget, setViewTarget, openCreate, openEdit } =  
  useCrudModal(emptyForm, toForm)
```

- `filtered` — `categories` -İ AXTARIŞ MƏTNİNƏ GÖRƏ FİLTRELƏYİR. `\ ${c.name} ${c.description}`` — ADI VƏ TƏSVİRİ BİRLƏŞDİRİR (İKİSİNDƏN BİRİNDƏ AXTARIŞ SÖZÜ VARSA TAPILSIN DEYƏ). `.toLowerCase()` — HƏR İKİ TƏRƏFİ KİÇİK HƏRFƏ ÇEVİRİR (BÖYÜK/KİÇİK HƏRF FƏRQİ OLMASIN DEYƏ). `.includes(...)` — BİR STRINGİN BAŞQA BİR STRINGİ EHTİVA EDİB-ETMƏDİYİNİ YOXLAYIR.
- `usePagination(filtered)` VƏ `useCrudModal(emptyForm, toForm)` — YUXARIDA İZAH OLUNAN İKİ ÖZ HOOK-UMUZ, BURADA BİRLİKDƏ İŞLƏDİLİR.

```
const handleSubmit = async (e) => {  
  e.preventDefault()  
  try {  
    const payload = mapCategoryToApi(form)  
    if (editing) {  
      await updateMutation.mutateAsync({ id: editing.id, payload })  
    } else {  
      await createMutation.mutateAsync(payload)  
    }  
    setFormOpen(false)  
  } catch {  
    // error already toasted by the global mutation cache  
  }  
}
```

- `mapCategoryToApi(form)` — FORMA DATASINI API FORMATINA ÇEVİRİR (HİSSƏ 8-Ə BAXIN).

- `if (editing) {...} else {...}` — `editing` `null` DEYİLSƏ (YƏNİ `openEdit` -DƏN GƏLİBSƏ), YENİLƏMƏ MUTASIYASINI, YOXSA (`openCreate` -DƏN GƏLİBSƏ) YARATMA MUTASIYASINI ÇAĞIRIR.
- `updateMutation.mutateAsync({ id: editing.id, payload })` — `mutateAsync` — `useMutation` - UN QAYTARDIĞI OBYEKTİN BİR METODUDUR, MUTASIYANI BAŞLADIR VƏ **PROMISE QAYTARIR** (`mutate` DA VAR, AMMA O PROMISE QAYTARMIR — BİZ `await` EDƏ BİLMƏK ÜÇÜN `mutateAsync` İŞLƏDİRİK).
- `catch {}` — BOŞ CATCH BLOKU: MUTASIYA XƏTA VERSƏ (SERVERDƏN 400/500 GƏLSƏ), BURAYA DÜŞÜR, AMMA HEÇ NƏ ETMİR — ÇÜNKİ TOAST ARTIQA QLOBAL OLARAQ GÖSTƏRİLİB. TƏK MƏQSƏD: `setFormOpen(false)` SƏTRİNİN İŞƏ DÜŞMƏMƏSİNİ TƏMİN ETMƏK (XƏTA OLANDA MODAL AÇIQA QALSIN, İSTİFADƏÇİ YENİDƏN CƏHD EDƏ BİLSİN).

```
const confirmDelete = async () => {
  try {
    await deleteMutation.mutateAsync(deleteTarget.id)
  } catch {
  } finally {
    setDeleteTarget(null)
  }
}

const submitting = createMutation.isPending || updateMutation.isPending
```

- `confirmDelete` — `ConfirmModal` -DA "TƏSDİQLƏ" BASILANDA ÇAĞIRILIR. `finally { setDeleteTarget(null) }` — İTƏR UĞURLU, İTƏR UĞURSUZ, `ConfirmModal` -I BAĞLAYIR (SİLİNMƏ XƏTASINDA MODAL AÇIQA SAXLAMAGIN MƏNASI YOXDUR, SADƏCƏ TOAST KİFAYƏTDİR).
- `submitting` — İKİ MUTASIYADAN (`create` / `update`) HANSISA HAZIRDA "İŞLƏYİRSƏ" (`isPending`), `true` OLUR — DÜYMƏNİ DEAKTİV ETMƏK VƏ "Göndərilir..." YAZDIRMAQ ÜÇÜN İSTİFADƏ OLUNUR.

JSX HİSSƏSİ (QISA):

```
<Button icon={Plus} onClick={() => openCreate()}>Yeni Kateqoriya</Button>

{loading && <Loading />}

<Table columns={columns} minWidth={720}>
  {paged.map((item, idx) => (
    <tr key={item.id}>
      <td>{(page - 1) * pageSize + idx + 1}</td>
      <td><Thumbnail imageUrl={item.imageUrl} image={item.image} color={item.color} /></td>
      ...
      <td><ActionMenu onView={() => setViewTarget(item)} onEdit={() => openEdit(item)}
onDelete={() => setDeleteTarget(item)} /></td>
    </tr>
  ))}
  {!loading && paged.length === 0 && <TableEmptyRow colSpan={columns.length} />}
</Table>
```

- `(page - 1) * pageSize + idx + 1` — CƏDVƏLDƏKİ "SIRA NÖMRƏSİ"Nİ HESABLAYIR (MƏS. 2-Cİ SƏHİFƏDƏ, PAGESIZE=5, 3-CÜ ELEMENT (idx=2) → $(2-1)*5 + 2 + 1 = 8$).
- `!loading && paged.length === 0` — YÜKLƏNMƏ BİTİB (Kİ, YÜKLƏNMƏ VAXTI "TAPILMADI" MESAJI YANLIŞ GÖRÜNMƏSİN) VƏ SİYAHİ HƏQİQƏTƏN BOŞDURSA, `TableEmptyRow` GÖSTƏRİLİR.

Formanın özü VƏ Modal-lar YUXARIDA İZAH OLUNAN `Modal / ConfirmModal` KOMPONENTLƏRİNİ İSTİFADƏ EDİR, HƏR `<input>` `value={form.X}` + `onChange={(e) => setForm((f) => ({ ...f, X: e.target.value })))` NÜMUNƏSİNİ TƏKRARLAYIR — BU, "**CONTROLLED INPUT**" DEYİLƏN ÜMUMİ REACT NÜMUNƏSİDİR: INPUTUN DƏYƏRİ TAM OLARAQ STATE-DƏN GƏLİR (`value={...}`), HƏR YAZILAN HƏRF `onChange` -İ İŞƏ SALIR, O DA STATE-İ YENİLƏYİR, YENİ STATE İSƏ INPUTUN GÖRÜNÜŞÜNÜ YENİLƏYİR — DÖVRƏ BELƏ TAMAMLANIR. `setForm((f) => ({ ...f, X: e.target.value })))` — `f` CARİ FORMA OBYEKTİDİR, `SPREAD ( ...f )` İLƏ BÜTÜN KÖHNƏ SAHƏLƏR SAXLANILIR, YALNIZ `x` SAHƏSİ YENİ DƏYƏRLƏ ƏVƏZ OLUNUR.

Campaigns.jsx və Products.jsx — Categories ilə FƏRQLƏR

Campaigns.jsx — QURULUŞ 1:1 EYNİDİR, SADƏCƏ:

- `name / description` ƏVƏZİNƏ `title / description` (KAMPANİYALARIN "ADI" API-DƏ `title` ADLANIR).
- SERVİS/ADAPTER FAYLLARI FƏRQLİDİR (`campaignService.js` , `adapters/campaign.js`), AMMA EYNİ ŞƏKİLDƏ İŞLƏYİR.

Products.jsx — EYNİ QURULUŞ + ƏLAVƏ MÜRƏKKƏBLİK:

```
const { data: categoryOptions = [] } = useQuery({
  queryKey: ['categories'],
  queryFn: () => listCategories().then((data) => data.map(mapCategoryFromApi)),
})
```

- İKİNCİ BİR `useQuery` ÇAĞIRIŞI VAR — MƏHSUL FORMASINDAKI "KATEQORİYA" DROPDOWN-U ÜÇÜN. `queryKey: ['categories']` — BU, `Categories.jsx` -İN İSTİFADƏ ETDİYİ **EYNİ AÇARDIR** — TANSTACK QUERY BUNU GÖRÜB İKİ AYRI SORĞU YERİNƏ BİR DƏFƏ ÇƏKİB PAYLAŞIR (CACHE DEDUPLICATION).

```
<Button icon={Plus} onClick={() => openCreate({ category_id: categoryOptions[0]?.id ?? ''
})}>Yeni Məhsul</Button>
```

- `openCreate` FUNKSİYASINA `{ category_id: ... }` VERİLİR (`useCrudModal` -DAKI `overrides` PARAMETRİ) — YENİ MƏHSUL FORMASI AÇILANDA, DROPDOWN-DA BİRİNCİ KATEQORİYA AVTOMATİK SEÇİLİ GƏLSİN DEYƏ.
- TİP (`Növ`) DROPDOWN-U `PRODUCT_TYPE_OPTIONS` -DAN QURULUR (HİSSƏ 9-A BAXIN), BADGE RƏNGİ `productTypeBadgeColor(item.type)` FUNKSİYASI İLƏ TƏYİN OLUNUR.

src/pages/protected/Orders/Orders.jsx — FƏRQLİ NÜMUNƏ

BU SƏHİFƏ `ActionMenu / useCrudModal` İŞLƏTMİR (BİLƏRƏKDƏN, LAYİHƏNİN DİZAYN QƏRARIDIR) — SADƏ "Göstər" DÜYMƏSİ + DETAL MODALI VAR, VƏ YARATMA/SİLMƏ YOXDUR, YALNIZ **STATUS DƏYİŞDİRMƏ**.

```
const { data: orders = [], isLoading: loading } = useQuery({
  queryKey: ['orders'],
  queryFn: () => listOrders().then((data) => data.map(mapOrderFromApi)),
})
const { data: statsData } = useQuery({
  queryKey: ['orderStats'],
  queryFn: getOrderStats,
```

```

}))
const stats = { ...emptyStats, ...statsData }

```

- **İKİ AYRI, MÜSTƏQİL** `useQuery` — SİFARİŞ SİYAHISI VƏ STATİSTİKA (KART DATASINI) AYRI-AYRI ÇƏKİLİR (BİR-BİRİNDƏN ASILI DEYİL, PARALEL İŞLƏYİR).
- `stats = { ...emptyStats, ...statsData }` — SPREAD-İN "SIRA İLƏ ÜSTƏLƏMƏ" XÜSUSİYYƏTİ BURADA MÜHÜM RİSK QARŞISINI ALIR ÜÇÜN İŞLƏDİLİR: `emptyStats` BÜTÜN AÇARLARI (TOTAL, PENDING VƏ S.) SIFIRLA TƏYİN EDİR, SONRA `...statsData` (API-DƏN GƏLƏN) HƏR HANSI MÖVCUD AÇARI ÜSTƏLƏYİR — ƏGƏR API BƏZİ AÇARLARI (MƏS. `CANCELLED`) QAYTARMASA, ONLAR `emptyStats` -DAN GƏLƏN `0` -DA QALIR (`undefined` OLMUR, EKRANDA "undefined" YAZILMIR).

```

const statusMutation = useMutation({
  mutationFn: ({ id, status }) => updateOrderStatus(id, status),
  onSuccess: async () => {
    await Promise.all([
      queryClient.invalidateQueries({ queryKey: ['orders'] }),
      queryClient.invalidateQueries({ queryKey: ['orderStats'] }),
    ])
    toast.success('Sifariş statusu yeniləndi')
  },
})

```

- `mutationFn: ({ id, status }) => updateOrderStatus(id, status)` — `mutate({id, status})` ÇAĞIRILANDA, BU FUNKSİYA O OBYEKTİ DESTRUCTURE EDİB, SERVİS FUNKSİYASINA İKİ AYRI ARQUMENT KİMİ ÖTÜRÜR.
- `Promise.all([...])` — İKİ ASİNXRON İŞİ (SİFARİŞLƏRİ VƏ STATİSTİKANI YENİLƏMƏ) **EYNI ANDA** BAŞLADIR VƏ HƏR İKİSİ BİTƏNƏ QƏDƏR GÖZLƏYİR (BİR-BİR, ARDICIL `await` ETMƏKDƏNSƏ DAHA SÜRƏTLİDİR, ÇÜNKİ PARALEL GEDİR).

```

const [selectedId, setSelectedId] = useState(null)
const selected = orders.find((o) => o.id === selectedId) ?? null

```

- **DİQQƏTLİ DİZAYN QƏRARI:** `selected` (AÇIQ MODALDAKİ SİFARİŞ) BİR "SNAPSHOT" (SEÇİLƏN ANDAKİ KOPYA) KİMİ SAXLANMIR — SADƏCƏ `selectedId` (ID-Sİ) SAXLANIR, `selected` -İN ÖZÜ İSƏ HƏR RENDER-DƏ `orders` SİYAHISINDAN **YENİDƏN TAPILIR** (`.find(...)`). BUNUN FAYDASI: STATUS DƏYİŞDİRİLİB `orders` SİYAHISI YENİLƏNƏNDƏ (`invalidateQueries` VASİTƏSİLƏ), `selected` DƏ AVTOMATİK TƏZƏ DATANI GÖSTƏRİR — ƏLAVƏ HEÇ BİR "MANUAL MERGE" KODU YAZMAĞA EHTİYAC QALMIR.

```

const updateStatus = (id, status) => {
  statusMutation.mutate({ id, status })
}

```

BURADA `mutate` İŞLƏDİLİR, `mutateAsync` YOX — ÇÜNKİ BU FUNKSİYA `await` / `try-catch` İLƏ NƏTİCƏNİ GÖZLƏMİR (SELECT ELEMENTİNİN `onChange` -İNDƏN ÇAĞIRILIR, "ATIB-UNUT" YANAŞMASI KİFAYƏTDİR — XƏTA/UĞUR HƏR HALDA QLOBAL OLARAQ TOAST-DA GÖRÜNƏCƏK).

`src/pages/protected/Users/Users.jsx` — ƏN SADƏ SƏHİFƏ

BU SƏHİFƏ TAMAMILƏ **OXUMAQ ÜÇÜNDÜR** — YARATMA, DÜZƏLTMƏ, SİLMƏ YOXDUR (BACKEND-DƏ BELƏ ENDPOINT-LƏR DƏ YOXDUR). YALNIZ:

```
const { data: users = [], isLoading: loading } = useQuery({
  queryKey: ['users'],
  queryFn: () => listUsers().then((data) => data.map(mapUserFromApi)),
})
const [selected, setSelected] = useState(null)
```

BİR `useQuery` , BİR "Göstər" DÜYMƏSİ/DETAL MODALI ÜÇÜN `selected` STATE-i (BURADA `Orders.jsx` -DƏKİ KİMİ ID-YƏ DAYALI DEYİL, ÇÜNKİ BU SİYAHİ HEÇ VAXT MUTASIYA OLUNMUR — DƏYİŞMƏYƏCƏK BİR SİYAHIDA "SNAPSHOT" SAXLAMAQ TAM TƏHLÜKƏSİZDİR). QALAN HƏR ŞEY — `usePagination` , AXTARIŞ FİLTİRİ, CƏDVƏL — EYNİ NÜMUNƏLƏRİ TƏKRARLAYIR.

Hissə 17: CSS Modules

HƏR KOMPONENTİN YANINDA (`Button.jsx` YANINDA `Button.module.css` KİMİ) BİR CSS FAYLI VAR. MİSAL:

```
/* Button.module.css */
.btn {
  padding: 10px 16px;
  border-radius: 8px;
}
.solid {
  background: var(--color-green);
}
```

```
import styles from './Button.module.css'
<button className={styles.btn}>...</button>
```

Nə baş verir? VITE, `.module.css` ADLI FAYLLARI GÖRƏNDƏ, HƏR KLAS ADINI (`.btn` , `.solid`) **UNİKAL BİR ADA** ÇEVİRİR (MƏSƏLƏN `Button-module__btn__x7K2m`) VƏ BUNLARI `styles` ADLI BİR JAVASCRIPT OBYEKTİ KİMİ İXRAC EDİR: `styles.btn` → `"Button-module__btn__x7K2m"` KİMİ BİR STRİNG. BELƏLİKLƏ, BAŞQA BİR FAYLDA DA `.btn` KLASI YAZSANIZ (MƏS. `Table.module.css` -DƏ), İKİSİ TOQQUŞMUR — HƏR BİRİ ÖZ FAYLINA MƏXSUS UNİKAL ADA MALİKDİR.

`var(--color-green)` **NƏDİR?** — `src/index.css` -DƏ TƏYİN OLUNAN CSS DƏYİŞƏNLƏRİDİR (CSS CUSTOM PROPERTIES):

```
:root {
  --color-green: #7cc576;
  --radius-sm: 8px;
}
```

`var(--color-green)` — BU DƏYİŞƏNİN DƏYƏRİNİ (`#7cc576`) OXUYUR. BÜTÜN RƏNGLƏR/ÖLÇÜLƏR BURADA BİR DƏFƏ TƏYİN OLUNUB, HƏR YERDƏ TƏKRAR YAZMAQ ƏVƏZİNƏ `var(--...)` İLƏ İSTİFADƏ OLUNUR — RƏNG SXEMİNİ DƏYİŞMƏK İSTƏSƏNİZ, TƏK BİR YERİ (`index.css`) DƏYİŞMƏK KİFAYƏTDİR.

Hissə 18: Lüğət

Termin	Mənası
Komponent	UI-nin bir hissəsini qaytaran funksiya (böyük hərflə başlayır)
Prop	Komponentə kənardan verilən "parametr"
State	Komponentin öz "yaddaşı" (<code>useState</code>) — dəyişəndə ekran yenilənir
Hook	<code>use</code> -la başlayan funksiya (<code>useState</code> , <code>useEffect</code> , öz hook-larımız)
JSX	HTML-ə bənzəyən, JavaScript-ə çevrilən sintaksis
Render	Komponentin JSX-ini hesablayıb ekrana çıxarmaq prosesi
Route	Bir URL-ə uyğun gələn səhifə təyinatı
Endpoint	Backend-in bir konkret ünvanı (məs. <code>/admin/categories</code>)
Payload	Serverə göndərilən data (body)
Adapter	API formatını UI formatına (və əksinə) çevirən funksiya
Query	TanStack Query-də data OXUMA əməliyyatı (<code>useQuery</code>)
Mutation	TanStack Query-də data YAZMA əməliyyatı (<code>useMutation</code>)
Cache	Yaddaşda saxlanan, təkrar sorğuya ehtiyacı azaldan data
Invalidate	Cache-dəki datanı "köhnəlmiş" elan edib yenidən çəkdirmək
Interceptor	Hər sorğu/cavabı "ələ keçirib" ortaq məntiq tətbiq edən axios funksiyası
Toast	Ekranın küncündə görünən, öz-özünə yox olan bildiriş qutucuğu
Debounce	İstifadəçi fəaliyyəti dayandırdıqdan sonra müəyyən gecikmə ilə davranmaq
Destructuring	Obyekt/massivdən sahələri ayrıca dəyişənlərə "çıxarmaq"
Spread (...)	Obyekt/massivi "açıb" başqasının içində tökmək
Rest (...)	Qalan bütün prop/elementləri bir yere yığmaq
Optional chaining (?.)	"Əgər varsa daxil ol", yoxdursa xəta vermədən <code>undefined</code> qaytar
Nullish coalescing (??)	Sol tərəf <code>null/undefined</code> -dursa sağ tərəfi istifadə et
Ternar operator (? :)	Qısa <code>if/else</code> : şərt ? doğrudursa : yanlışdırsa
Async/await	Asinxron (gecikən) işi "gözləyərək" ardıcıl kodmuş kimi yazmaq üsulu
Promise	"Gələcəkdə bitəcək iş" təmsil edən JavaScript obyektı
localStorage	Brauzerin, səhifə bağlansa belə itməyən yaddaşı

CSS Modules	Hər faylın klas adlarını avtomatik unikal edən CSS sistemi
Controlled input	Dəyəri tam React state-dən idarə olunan <code><input></code>
Custom hook	Öz yazdığımız, <code>use-</code> la başlayan, təkrarlanan məntiqi yığan funksiya
Portal	Bir elementi, React ağacındakı yerindən asılı olmayaraq, DOM-da başqa yerə "işinləmək" (<code>createPortal</code>)
Error Boundary	Uşaqlarındakı JS xətlərini tutub, bütün tətbiqin çökməsinin qarşısını alan class komponent
Lazy loading	Bir komponentin kodunu YALNIZ lazım olanda yükləmək (<code>React.lazy</code>)
Suspense	<code>lazy</code> komponent yüklənənə qədər nə göstəriləcəyini idarə edən React komponenti
Chunk	Build zamanı ayrılan kiçik JS fayl parçası

Son qeyd: Bu sənəd, kodun HAZIRKI vəziyyətini əks etdirir. Kod dəyişdikcə (yeni səhifə, yeni funksiya əlavə olunduqca) bu sənədin də yenilənməsi lazımdır ki, köhnəlməsin.